FarOS 操作系统开发手册

FarOS (Fitness & Health microkernel Real-time Operating System)
版本: v1.2 | 日期: 2026-10-23

目录

1. 系统概述
   · 1.1 项目背景
   · 1.2 系统架构
      · 1.2.1 各层详细说明
      · 1.2.2 Middleware Services 详细介绍
   · 1.3 核心技术特性
   · 1.4 应用场景
   · 1.5 源码目录与启动顺序
2. 快速开始
   · 2.1 5分钟上手流程
   · 2.2 验证固件运行
3. 开发环境搭建
   · 3.1 硬件环境
   · 3.2 软件环境
   · 3.3 编译与烧录
   · 3.4 编译目标说明
4. 内核编程指南
   · 4.1 任务管理
   · 4.2 消息队列
   · 4.3 定时器
   · 4.4 内存管理
5. 驱动开发指南
   · 5.1 I2C驱动
   · 5.2 GPIO驱动
   · 5.3 ADC驱动
6. 蓝牙开发指南
   · 6.1 BLE GATT服务开发
   · 6.2 标准健康Profile
7. 运动健康功能开发
   · 7.1 模块架构
      · 7.1.1 模块依赖关系
      · 7.1.2 编译控制
      · 7.1.3 服务启动与数据通路
      · 7.1.4 公共接口与用户档案
   · 7.2 传感器数据采集
      · 7.2.1 传感器抽象层

        · 7.2.2 传感器配置建议
    · 7.3 运动识别算法接口
        · 7.3.1 功能概述
    · 7.4 生理指标算法接口
        · 7.4.1 功能概述
        · 7.4.2 Far Beat核心概念参考
        · 7.4.3 API接口
    · 7.5 睡眠分析接口
        · 7.5.1 功能概述
    · 7.6 数据管理接口
        · 7.6.1 功能概述
        · 7.6.2 调用示例
    · 7.7 AT命令调试接口
    · 7.8 性能指标与优化建议
        · 7.8.1 算法性能指标
        · 7.8.2 系统资源占用
        · 7.8.3 功耗优化建议
8. 故障排除与FAQ
    · 8.1 编译问题
    · 8.2 烧录问题
    · 8.3 运行时问题
    · 8.4 运动健康模块问题
9. 附录
    · 附录A: 运动健康API速查表
    · 附录B: 错误码表
    · 附录C: 数据类型定义
    · 附录D: 文件清单
    · 附录E: 术语表

1. 系统概述

1.1 项目背景

FarOS 是面向运动健康垂直领域的专用型微内核操作系统，依托国家重点专项《不同经济区域主动健康技术创新应用示范》（项目编号：2023YFC3606100）立项研发。

系统基于开源微内核基础架构完成场景化迭代与定制优化，聚焦运动健康智能设备专属应用场景，重点攻克多品类、多厂商智能健康设备跨域互操作核心难题。

1.2 系统架构

```
+-----------------------------------------+
|  Application Layer                      |
|  (brandy_standard, brandy_recovery, wstp)|
+-----------------------------------------+
|  Middleware Services                    |
|  + Health Service (运动健康服务)          |
|    - far_sensor    传感器抽象层          |
|    - far_activity  运动识别              |
|    - far_physio    生理指标              |
|    - far_sleep     睡眠分析              |
|    - far_health_data 数据管理            |
|  + Bluetooth Service (蓝牙服务)          |
|  + DFX (诊断与调试)                      |
|  + PM (电源管理)                         |
|  + FS (文件系统)                         |
|  + NV (非易失存储)                       |
|  + Security (安全子系统)                 |
+-----------------------------------------+
|  Kernel Layer                           |
|  + OSAL (操作系统抽象层)                   |
|  + LiteOS / FreeRTOS                    |
|  + DPAL (设备平台抽象层)                   |
+-----------------------------------------+
|  Driver Layer                           |
|  + HAL (硬件抽象层)                       |
|  + Device Drivers                       |
+-----------------------------------------+
|  Hardware                               |
|  + Brandy SoC (RISC-V 32-bit)           |
+-----------------------------------------+
```

1.2.1 各层详细说明

Application Layer（应用层）

应用层面向具体业务场景，提供两种运行模式：

运行模式	路径	说明
brandy_standard	application/brandy/brandy_standard/	标准模式，完整功能（内核+驱动+服务+AT命令）
brandy_recovery	application/brandy/brandy_recovery/	恢复模式，用于固件升级失败后的系统恢复
wstp	application/wstp/at_service/	AT命令服务，支持通过串口进行系统调试和控制

应用层通过 CMSIS-RTOS2 标准接口与内核交互，具备以下特点：

- 任务化设计：系统任务、蓝牙任务、AT命令任务、健康算法任务等独立运行
- 消息队列通信：各模块间通过消息队列解耦通信
- 事件驱动：按键、触摸、传感器中断等事件触发回调处理

Middleware Services（中间件服务层）

中间件是 FarOS 的核心价值层，面向运动健康场景提供专用服务：

服务模块	路径	核心功能
Health Service	middleware/services/health/	运动健康服务（本手册重点），含传感器抽象、运动识别、生理指标、睡眠分析、数据管理
Bluetooth Service	protocol/bt/	BLE 5.3 / BREDR 协议栈，支持GATT、GAP、A2DP、HFP等Profile，负责设备间无线通信
DFX	middleware/chips/brandy/dfx/	诊断与调试框架：多级别日志系统（SDT/压缩/文件）、异常现场dump、panic自动重启、串口/HSO在线调试
PM	middleware/chips/brandy/pm/	电源管理：深睡/浅睡状态机、tickless空闲调度、动态电压频率调节（DVFS）、唤醒源管理
FS	middleware/utils/fs/	文件系统：FAT（兼容Windows）、YAFFS2（NANDFlash专用）、VFS虚拟文件系统，支持EMMC/NANDFlash/NorFlash多介质
NV	middleware/utils/nv/	非易失存储：键值对持久化数据库，磨损均衡、掉电保护、CRC校验，用于保存用户配置和校准参数
Security	middleware/utils/sec_boot/	安全子系统：安全启动链、OTA固件签名验证、AES/SM4硬件加密、真随机数生成器（TRNG）

中间件服务层采用**模块化+服务化**架构设计，各服务独立编译、按需链接，具备以下开发特性：

- **统一初始化框架**：所有服务通过 service_init() 接口注册到系统启动流程，按依赖顺序自动初始化
- **消息总线通信**：服务间通过 osal_msg_queue 解耦通信，避免直接调用导致的耦合
- **动态加载能力**：支持 dlopen 式动态服务加载，非核心服务可按场景裁剪
- **标准化接口**：每个服务对外提供 uapi_<service>_init/enable/disable 统一 API 模式

中间件服务开发流程：

```
1. 创建服务目录：middleware/services/<my_service>/
2. 实现服务接口：uapi_my_service_init() / uapi_my_service_enable()
3. 注册初始化钩子：SERVICE_INIT(my_service_init)
4. 配置编译开关：在 config.py 中添加 'SUPPORT_MY_SERVICE': 'YES'
5. 链接服务库：在 CMakeLists.txt 中添加 target_link_libraries()
```

1.2.2 Middleware Services 详细介绍

Bluetooth Service（蓝牙服务）

蓝牙服务是运动健康设备与外部世界通信的核心通道。FarOS 内置完整的 BTS（Bluetooth Stack）协议栈，支持 BLE 5.3 和经典蓝牙 BREDR 双模工作。在运动健康场景中，蓝牙承担以下关键职责：

职责	说明
数据同步	将运动记录、睡眠数据、心率曲线同步到手机APP（如华为运动健康、Keep）
实时数据广播	通过 BLE HRS Profile 向跑步机、骑行台等外设实时发送心率数据
固件升级	支持 BLE OTA（Over-The-Air）无线固件升级，无需有线连接
外设连接	作为 GATT Client 连接心率带、踏频器、功率计等第三方传感器
音频传输	通过 A2DP 连接蓝牙耳机，播放运动指导语音或来电提醒

DFX（诊断与调试框架）

DFX（Design For X，面向全生命周期的设计）是保障系统稳定性和可维护性的关键基础设施。在可穿戴设备中，由于用户无法直接观察设备内部状态，完善的诊断能力尤为重要：

功能模块	说明	健康场景应用
日志系统	支持SDT（串口实时输出）、压缩日志、文件日志三种模式	远程分析用户运动数据异常原因
异常dump	系统崩溃时自动保存寄存器、调用栈、内存现场	定位算法任务崩溃或内存越界问题
panic处理	致命错误时自动重启并保存现场，避免设备变砖	保障长时间佩戴的可靠性
在线调试	支持HSO（高速串口）和J-Link在线调试	开发阶段实时查看传感器原始数据
性能统计	CPU负载、任务调度、内存使用实时监控	优化算法任务的资源占用

PM（电源管理）

电源管理是穿戴设备续航的核心。FarOS 的 PM 模块采用状态机驱动设计，根据系统活动状态自动切换功耗模式：

功耗模式	CPU状态	典型功耗	进入条件	退出条件
运行模式	全速运行	5-15mA	有算法任务在处理	系统进入空闲
空闲模式	低频运行	2-5mA	无活跃任务，tickless触发	中断/定时器到期
浅睡模式	部分时钟关闭	0.5-2mA	浅睡投票通过	GPIO中断/RTC到期
深睡模式	主核关闭，ULP运行	0.1-0.5mA	深睡投票通过，无睡眠否决	ULP唤醒源触发

PM 模块的关键设计：

- **睡眠投票机制**：各模块（传感器、蓝牙、健康算法）通过投票表达是否允许睡眠
- **Tickless调度**：空闲时停止系统节拍计时器，消除周期性定时器唤醒
- **唤醒源管理**：GPIO中断（按键/触摸）、RTC定时器、BLE事件、传感器FIFO满等均可作为唤醒源

FS（文件系统）

文件系统为运动健康数据提供持久化存储能力。FarOS 采用 VFS（Virtual File System）架构，上层应用使用统一接口，底层适配不同存储介质：

文件系统类型	适用介质	特点	健康场景应用
FAT	EMMC / SD卡	兼容Windows，便于导出数据到PC	运动记录文件导出
YAFFS2	NANDFlash	专为NAND优化，支持坏块管理	大量运动历史数据存储
ROMFS	NorFlash	只读，用于固件和资源文件	固件镜像存储
VFS	抽象层	统一文件操作接口（open/read/write/close）	应用层透明访问

NV（非易失存储）

NV（Non-Volatile）模块提供键值对形式的持久化存储，用于保存需要掉电保护的配置数据和校准参数：

数据类别	示例	说明
用户配置	用户档案（性别/年龄/身高/体重）、闹钟设置、显示偏好	用户手动设置的个性化参数
校准数据	IMU零偏、PPG LED校准、心率区间	出厂或运行时校准的参数
运行状态	今日步数、累计距离、当前身体电量	需要跨重启保持的动态数据
系统配置	蓝牙配对信息、WiFi凭据、OTA版本	系统级关键配置

NV 模块的核心特性：

- **磨损均衡**：对Flash存储区域进行轮询写入，延长存储寿命
- **掉电保护**：写操作采用双备份+原子切换，确保掉电不损坏数据
- **CRC校验**：每条记录附带 CRC 校验码，用于发现掉电或位翻转造成的数据损坏
- **版本兼容**：支持数据格式升级时的向后兼容

Security（安全子系统）

安全子系统保障运动健康数据的隐私性和设备运行的完整性：

安全功能	说明	健康场景意义
安全启动	启动时逐级校验Bootloader、固件签名	防止恶意固件刷入，保护用户健康数据
OTA签名验证	固件升级前验证数字签名	确保升级包来源可信
AES/SM4加密	硬件加速的对称加密	加密存储敏感健康数据（心率、睡眠）
TRNG	真随机数发生器	生成安全的密钥和会话令牌

Kernel Layer（内核层）

内核层提供操作系统基础能力，采用**双内核+抽象层**设计：

组件	路径	功能说明
OSAL	kernel/osal/	操作系统抽象层，提供统一的任务/队列/定时器/内存/中断API，屏蔽LiteOS与FreeRTOS差异
LiteOS	kernel/liteos/liteos_v207.1.0/	华为 LiteOS v207.1.0，轻量级实时内核，默认用于 brandy-mcu-bt 目标
FreeRTOS	kernel/freertos/FreeRTOS/	FreeRTOS 内核，可选替代方案，支持 CMSIS-RTOS2 接口
DPAL	kernel/dpal/	设备平台抽象层，提供 POSIX-like 文件操作、设备管理、MTD接口
non_os	kernel/non_os/	裸机运行时，用于 bootloader 等无操作系统场景

内核层关键特性：

- **任务调度**：基于优先级的抢占式调度，支持同优先级时间片轮转
- **内存管理**：多内存池设计（DTCM/L2RAM/PSRAM），支持静态/动态内存分配
- **中断管理**：支持中断嵌套，中断与任务间通过信号量/事件同步
- **低功耗**：Tickless机制，在空闲时停止系统节拍计时器，降低功耗

Driver Layer（驱动层）

驱动层采用 **HAL（硬件抽象层）+ Driver（设备驱动）** 双层架构：

层级	路径	功能
HAL	drivers/drivers/hal/	寄存器级操作封装，定义硬件IP的寄存器地址、位域、访问接口
Driver	drivers/drivers/driver/	设备驱动实现，向上层提供统一的标准设备接口
Board	drivers/boards/brandy_evb/	板级支持包，定义GPIO引脚复用、外设连接关系
Chip	drivers/chips/brandy/	芯片级初始化代码，时钟、复位、电源域配置

主要驱动类型：

驱动类别	具体驱动	健康设备用途
通信接口	I2C、SPI、UART、QSPI	连接IMU、PPG、GPS等传感器
存储	EMMC、NANDFlash、NorFlash、PSRAM	运动记录、日志、固件存储
显示	LCD (QSPI/MIPI)、Touch	智能手表屏幕与触摸交互
电源	PM、LDO、RTC	电池管理、低功耗计时
安全	AES、TRNG、PKE	健康数据加密保护
音频	I2S、PDM、PCM	语音助手、音频提示

Hardware（硬件层）

FarOS 主要面向 **Brandy SoC** 平台进行开发和优化：

硬件规格	参数
CPU架构	RISC-V RV32IMAC (riscv70 core)
主频	最大 480MHz
内存	DTCM (数据紧耦合存储器)、L2RAM、外部PSRAM
存储	片上NorFlash + 外部EMMC/NANDFlash
通信	BLE 5.3 / BREDR、WiFi 6 (可选)、UART/SPI/I2C
安全	硬件AES/SM4、真随机数发生器、安全启动
低功耗	支持深睡/浅睡/掉电模式，ULP子系统独立运行

Brandy SoC 面向可穿戴设备的关键设计：

- **异构多核**：A核（应用处理）+ M核（蓝牙协议栈）+ ULP核（始终在线传感器采集）
- **传感器直连**：IMU、PPG等传感器可通过I2C/SPI直接连接，支持DMA传输
- **Always-On域**：ULP子系统在主核深睡时仍可运行，适合计步、手势唤醒等功能

· **低功耗蓝牙**：集成BLE 5.3射频，支持长距离编码PHY（Coded PHY）

1.3 核心技术特性

特性	描述
超低功耗运行	精简系统冗余开销，优化算法与数据传输逻辑，延长设备续航
高安全高可靠	微内核隔离优势，完善权限管控、异常容错与风险预警机制
广谱设备兼容	适配感知类、电机健身类多品类终端，支持多品牌多型号设备快速接入
轻量化易部署	系统资源占用低，适配中低端硬件，提供标准化轻量化交钥匙解决方案

1.4 应用场景

FarOS 把传感器采集、运动与生理算法、本地存储和蓝牙同步放在同一套服务里，面向下面四类落地场景：

场景	设备形态	主要使用的服务
大众主动健康监测	手环、手表等全天佩戴设备	far_sensor 低采样计步，far_physio 心率/压力/身体电量，far_sleep 夜间分期，PM 负责深睡续航
智能化科学健身	跑步、骑行、力量训练终端	far_activity 识别步行/跑步/骑行，far_physio 计算卡路里、VO2max、EPOC 训练负荷和恢复时间
运动健康数据管控	需要导出或同步历史记录的设备	far_health_data 以 NV + 文件系统保存运动和睡眠记录，BLE GATT 把记录同步到手机
智慧体育场景建设	场馆、校园或多设备联动终端	BTS 同时作为 GATT Server 上报本机数据，并作为 GATT Client 接入心率带、踏频器等外设

1.5 源码目录与启动顺序

仓库按层次拆分，运动健康相关代码集中在中间件服务和 AT 命令两处：

目录	内容
application/brandy/brandy_standard/	标准模式入口。main.c 的 app_main() 依次完成内核、芯片、板级和服务初始化
application/brandy/brandy_recovery/	恢复模式，固件升级失败后的最小系统
application/brandy/brandy_standard/test_data/	活动识别回放样本：imu_sit.csv、imu_walk.csv、imu_run.csv
application/wstp/at_service/	AT 命令服务。健康命令在 app/app_health_at_process.c，并注册进 app_cmd_manager.c
middleware/services/health/	运动健康服务实现
include/middleware/services/health/	运动健康服务对外头文件
include/middleware/services/bts/	蓝牙 Host API（GATT Server/Client、LE GAP、经典蓝牙 Profile）
protocol/bt/host/bt/bts/	蓝牙协议栈与 Profile 示例，BAS、DIS 示例在 bt_sample/ble_profile/profiles/
kernel/	OSAL、LiteOS、FreeRTOS、DPAL、裸机运行时
drivers/	HAL、设备驱动、boards/brandy_evb 板级支持、chips/brandy 芯片初始化
middleware/chips/brandy/	Brandy 上的 DFX、电源管理和异常处理
middleware/utils/	文件系统、NV、升级、安全启动等公共组件
bootloader/	SSB / Flashboot
build/script/build.py	配置、编译、烧录、日志入口
build/config/target_config/brandy/config.py	Brandy 编译目标定义

标准模式的启动顺序写在 application/brandy/brandy_standard/main.c：

```
app_main()
  1. kernel_init()        内核与内存
```

```
2. chip_init()          时钟、复位、芯片外设
3. board_init()         板级引脚与存储介质
4. service_init()       日志、分区、OTA、NV、DFX、电源管理、AT 命令注册
5. thread_pri_init()    创建系统任务
6. osKernelStart()      启动调度
```

service_init() 会注册 AT 命令表，但不会调用 uapi_health_service_init()。运动健康服务要在串口执行 AT^HEALTHINIT，或由应用在任务里显式调用该函数后才开始采样和算法调度。

2. 快速开始

本章面向首次接触 FarOS 的开发者，提供一个从环境准备到运行首个固件的极简路径。熟悉嵌入式开发的开发者可跳过本章，直接阅读第3章。

2.1 5分钟上手流程

按照以下步骤，可在5分钟内完成首次编译和烧录：

步骤1：克隆代码仓库

```
git clone <仓库地址> FarOS
cd FarOS
```

步骤2：安装工具链依赖

- Windows：安装 Linx LLVM RISC-V 交叉编译器，并添加到系统 PATH
- Python 3.8+（用于构建脚本）
- CMake 3.16+
- J-Link 驱动（用于烧录和调试）

步骤3：一键配置、编译、烧录

```
# 配置+编译+烧录（首次运行）
python build/script/build.py -t brandy-mcu-bt -c -b -f
```

步骤4：查看运行日志

```
# 新开终端窗口执行
python build/script/build.py -t brandy-mcu-bt -l
```

启动阶段串口应出现分区和升级模块的初始化打印，例如：

```
partition init success
upg init success
```

运动健康服务在启动时不会自动运行。串口发送 AT^HEALTHINIT 后，应看到服务自己的打印：

```
[health_service] Initializing FarOS Health Service v1.0.0
[health_service] Health task started
[health_service] FarOS Health Service initialized successfully
[HEALTH] Init OK
```

2.2 验证固件运行

固件启动后，可通过 AT 命令验证运动健康服务是否正常工作：

```
# 在串口终端输入
AT^HEALTHINIT
# 预期输出:
# [HEALTH] Init OK

AT^HEALTHVER
# 预期输出:
# [HEALTH] Version: 1.0.0

AT^HEALTHSTATUS
# 预期输出:
# [HEALTH] Status:
#   IMU power: <uA>
#   PPG power: <uA>
#   Total sensor power: <uA>
#   Storage: <used>/<total> bytes
```

AT^HEALTHSTATUS 里的传感器功耗单位是微安（uA），由 uapi_sensor_get_power_estimate() 返回。存储占用来自 uapi_health_data_get_storage_usage()。

如果以上命令均返回预期结果，说明 FarOS 运动健康服务已成功运行。接下来可以：

- 阅读 第3章 深入了解开发环境配置
- 阅读 第7章 学习运动健康功能开发
- 阅读 第5章 开发自定义传感器驱动

3. 开发环境搭建

3.1 硬件环境

开发 FarOS 应用需要准备以下硬件设备。Brandy EVB 开发板是官方推荐的硬件平台，集成了 Brandy SoC 及其外围电路，便于快速验证和调试。

项目	要求	说明
目标芯片	Brandy SoC (RISC-V RV32IMAC)	FarOS 的主要目标平台，集成 BLE 5.3 射频
开发板	Brandy EVB	官方评估板，集成调试接口、USB转串口、按键、LED
调试器	J-Link / J-Link OB	用于程序下载和在线调试，支持 SWD 接口
串口	USB转TTL (115200bps, 8N1)	用于查看系统日志和 AT 命令交互

硬件连接方式：

1. 使用 USB 线连接 Brandy EVB 到 PC（提供供电和串口通信）
2. 使用 J-Link 连接开发板的 SWD 接口（用于调试和烧录）
3. 确保串口波特率设置为 115200，数据位 8，停止位 1，无校验

3.2 软件环境

FarOS 采用 CMake 构建系统，支持 Windows 和 Linux 双平台开发。推荐使用 Windows 环境，因为部分工具链（如 Linx LLVM）在 Windows 上有更好的原生支持。

项目	版本	说明
操作系统	Windows 10/11 或 Linux Ubuntu 20.04+	Windows 推荐，工具链原生支持更好
编译工具链	cc_riscv32_musl_fp_win (Linx LLVM)	RISC-V 32位交叉编译器，支持 musl libc 和硬件浮点
构建系统	CMake 3.16+	跨平台构建配置
Python	3.8+	构建脚本和代码生成工具依赖
Git	2.30+	版本管理和子模块拉取

环境变量配置（Windows）：

将以下路径添加到系统 PATH 环境变量：

- tools/bin/compiler/linx/linx_170/bin/ — 编译器路径
- tools/bin/jlink_tool/ — J-Link 工具路径

3.3 编译与烧录

FarOS 提供统一的 Python 构建脚本，覆盖配置、编译、烧录、日志查看全流程。初次编译前需要先生成构建配置（仅第一次），后续可直接编译。

完整开发流程：

```
# 步骤1：配置编译目标（首次编译或切换目标时执行）
# -t 指定目标，brandy-mcu-bt 是标准MCU+蓝牙目标
# -c 表示仅配置，不编译
python build/script/build.py -t brandy-mcu-bt -c

# 步骤2：编译整个项目
# 编译完成后，固件输出在 output/brandy/acore/<目标名>/ 目录下
python build/script/build.py -t brandy-mcu-bt -b

# 步骤3：烧录固件到开发板（通过J-Link）
# 确保J-Link已连接且开发板已上电
python build/script/build.py -t brandy-mcu-bt -f
```

```
# 步骤4：查看串口日志（新开终端窗口执行）
# 可实时查看系统启动日志和 AT 命令输出
python build/script/build.py -t brandy-mcu-bt -l
```

常用编译命令速查：

命令	功能
build.py -t brandy-mcu-bt -c	生成构建配置
build.py -t brandy-mcu-bt -b	编译项目
build.py -t brandy-mcu-bt -f	烧录固件
build.py -t brandy-mcu-bt -l	查看串口日志
build.py -t brandy-mcu-bt -c -b -f	配置+编译+烧录（一键完成）
build.py -t brandy-mcu-bt -j8	使用8线程并行编译
build.py -t brandy-mcu-bt --clean	清理编译输出

编译输出文件说明：

编译完成后，产物位于 output/brandy/acore/<目标名>/。当前 Brandy 目标对应的镜像名称如下：

目标	镜像	说明
brandy-mcu-bt	应用镜像	标准 MCU + 蓝牙固件，烧录和 J-Link 调试使用这一份
brandy-mcu-bt-r	recovery 镜像	恢复模式固件
brandy-ssb	ssb.bin	安全启动镜像
pack_brandy_mcu_bt	组合包	按 SSB、恢复镜像、应用镜像的顺序打包

同一目录下还会生成 ELF（在线调试）、map（符号地址）和反汇编列表。打包脚本里的历史路径仍使用 application.bin、recovery.bin、ssb.bin 这组文件名，见 build/config/target_config/brandy/build_brandy_update.py。

3.4 编译目标说明

目标定义在 build/config/target_config/brandy/config.py。日常开发使用 brandy-mcu-bt。

目标	内核与板级	包含的能力
brandy-mcu-bt	LiteOS，board=evb，模板 target_standard_brandy_application_template	BLE、BREDR、AT 命令、蓝牙管理 AT、DFX、NV、FAT/YAFFS2、电源管理、OTA 宏。配置注释标明该目标不含 GUI、多媒体和运动算法大包，运动健康服务需单独打开 SUPPORT_HEALTH_SERVICE
brandy-mcu-bt-r	LiteOS，恢复模板 target_brandy_recovery_template	分区、文件系统、电源管理和升级所需的最小组件
brandy-ssb	裸机安全启动，target=ssb	安全启动、code loader、NAND/EMMC 启动
brandy-mcu-bt-ota	别名，基目标为 brandy-mcu-bt	兼容旧的 OTA 目标名
brandy-mcu-bt-ota-r	别名，基目标为 brandy-mcu-bt-r	兼容旧的恢复 OTA 目标名

target_group 中的 pack_brandy_mcu_bt 成员顺序是 brandy-ssb、brandy-mcu-bt-r、brandy-mcu-bt。整机烧录需要这三份镜像都编译成功。

4. 内核编程指南

4.1 任务管理

FarOS 通过 OSAL（操作系统抽象层）提供统一的多任务管理能力，底层可适配 LiteOS 或 FreeRTOS。任务管理是嵌入式实时系统的核心，合理的任务划分直接影响系统的响应速度和资源利用率。

任务设计原则：

原则	说明	示例
职责单一	每个任务只负责一类功能	健康算法任务只处理传感器数据和算法计算
优先级合理	实时性要求高的任务分配更高优先级	传感器中断处理 > 算法计算 > 数据存储
避免忙等待	使用延时或事件等待释放CPU	osDelay() 或 osSemaphoreAcquire()
栈空间适度	根据任务调用深度分配栈空间	简单任务 1KB，复杂任务 4KB+

常用任务管理API：

函数	功能
osThreadNew(func, arg, attr)	创建新任务
osThreadTerminate(id)	终止指定任务
osThreadSetPriority(id, prio)	设置任务优先级
osThreadGetId()	获取当前任务ID
osDelay(ticks)	任务延时（毫秒）

```
#include "cmsis_os2.h"

/* 创建任务 */
osThreadId_t task_id = osThreadNew(my_task_func, NULL, NULL);

/* 任务函数 */
static void my_task_func(void *arg)
{
    while (1) {
        /* 任务逻辑 */
        osDelay(100);  /* 延时100ms */
    }
}
```

4.2 消息队列

消息队列是 FarOS 中模块间解耦通信的主要机制。通过消息队列，不同任务可以安全地交换数据，无需直接访问对方的内存空间。这种设计提高了系统的模块化和可维护性，尤其在运动健康场景中，传感器任务、算法任务、存储任务之间大量依赖消息队列进行数据传递。

消息队列的工作模式：

模式	说明	适用场景
阻塞接收	队列为空时任务挂起等待	消费者任务，如健康算法处理任务
非阻塞接收	队列为空时立即返回错误	需要轮询检查的场景
超时接收	等待指定时间后返回	需要兼顾响应和避免死锁的场景
覆盖发送	队列满时覆盖最旧消息	实时性优先，允许丢数据的场景

消息队列API：

函数	功能
osal_msg_queue_create(name, count, &id, flags, size)	创建消息队列

函数	功能
osal_msg_queue_write_copy(id, data, len, timeout)	发送消息（拷贝模式）
osal_msg_queue_read_copy(id, buf, &len, timeout)	接收消息（拷贝模式）
osal_msg_queue_delete(id)	删除消息队列

```
#include "soc_osal.h"

/* 创建队列：队列名、消息数量、队列ID、标志位、单条消息大小 */
osal_msg_queue_create("my_queue", 10, &queue_id, 0, sizeof(my_msg_t));

/* 发送消息：队列ID、数据指针、数据长度、超时时间（0=不等待） */
osal_msg_queue_write_copy(queue_id, &msg, sizeof(msg), 0);

/* 接收消息：永久阻塞等待，直到有消息到达 */
osal_msg_queue_read_copy(queue_id, &msg, &size, OSAL_MSGQ_WAIT_FOREVER);
```

4.3 定时器

FarOS 提供高精度定时器支持，基于 TCXO（温度补偿晶体振荡器）时钟源，确保在宽温度范围内保持稳定的时间精度。定时器系统支持微秒级和毫秒级延时，以及系统运行时间的获取，是传感器采样时序控制、算法周期性处理和低功耗管理的基础。

定时器类型对比：

类型	精度	最大延时	特点
Busy延时	微秒级	无限制	忙等待，CPU占用率高，适用于极短延时
OS延时	毫秒级	无限制	任务挂起，释放CPU，推荐用于任务延时
软件定时器	毫秒级	取决于实现	基于Tick的回调定时器，适合周期性任务
硬件定时器	微秒级	取决于分频	基于硬件定时器中断，精度最高

低功耗场景注意事项：

- 使用 osDelay() 代替忙等待，使系统有机会进入低功耗模式
- 长周期定时任务可配合 tickless 机制，在空闲时停止系统节拍
- 传感器采样定时建议使用硬件定时器中断，精确控制采样间隔

```
#include "tcxo.h"

/* 忙等待延时：适用于短于1ms的精确延时 */
uapi_tcxo_delay_us(100);   /* 延时100us */

/* 任务挂起延时：释放CPU，推荐用于任务调度 */
uapi_tcxo_delay_ms(100);   /* 延时100ms */

/* 获取系统运行时间：从开机起的累计时间 */
uint64_t ms = uapi_tcxo_get_ms();  /* 毫秒 */
uint64_t us = uapi_tcxo_get_us();  /* 微秒 */

/* 计算时间间隔 */
uint64_t start = uapi_tcxo_get_ms();
/* ... 执行某操作 ... */
uint64_t elapsed = uapi_tcxo_get_ms() - start;
```

4.4 内存管理

FarOS 支持多种内存分配策略，以适应不同场景的需求。在资源受限的嵌入式环境中，合理的内存管理是系统稳定运行的关键。运动健康算法涉及大量传感器数据缓冲（如IMU原始数据、PPG波形），对内存管理有较高要求。

内存区域划分：

区域	用途	特点
DTCM	数据紧耦合存储器	访问速度最快，容量较小（几十KB），适合频繁访问的数据
L2RAM	二级缓存RAM	容量中等，速度较快，适合任务栈和中等数据缓冲
PSRAM	外部伪静态RAM	容量最大（可达8MB+），速度较慢，适合大缓冲区和文件缓存
Flash	代码和常量存储	只读或擦写受限，适合固件和配置数据

内存管理最佳实践：

实践	说明
优先使用静态分配	在编译时确定内存需求，避免运行时碎片和分配失败
使用安全内存函数	memset_s / memcpy_s 带边界检查，防止缓冲区溢出
及时释放动态内存	避免内存泄漏，尤其是在周期性创建临时缓冲的场景
注意内存对齐	DMA操作通常要求16字节对齐，使用 memalign(16, size)

```
/* 动态内存分配：从系统堆中分配 */
void *ptr = malloc(size);
if (ptr != NULL) {
    /* 使用内存... */
    free(ptr);  /* 使用完毕后释放 */
}

/* 对齐内存分配：DMA传输要求16字节对齐 */
void *dma_buf = memalign(16, buf_size);
if (dma_buf != NULL) {
    /* DMA传输... */
    free(dma_buf);
}

/* 安全内存操作：带目标缓冲区大小检查，防止溢出 */
memset_s(dst, dst_size, 0, dst_size);          /* 安全清零 */
memcpy_s(dst, dst_size, src, src_size);        /* 安全拷贝 */
```

5. 驱动开发指南

FarOS 驱动层采用 **HAL（硬件抽象层）+ Driver（设备驱动）** 双层架构。HAL 层封装了寄存器级操作，提供与具体芯片版本无关的统一接口；Driver 层基于 HAL 实现具体设备的功能逻辑。这种分层设计使得上层应用和中间件无需关心底层硬件细节，便于跨平台移植。

驱动开发的一般流程：

1. **引脚配置**：通过板级支持包（Board）定义 GPIO 复用和外设连接
2. **时钟使能**：在芯片初始化时使能对应外设的时钟
3. **HAL初始化**：配置外设的寄存器参数（波特率、模式、中断等）
4. **Driver注册**：将设备注册到系统设备框架，提供标准文件操作接口
5. **应用调用**：上层通过统一 API 访问设备

5.1 I2C驱动

I2C（Inter-Integrated Circuit）是运动健康设备中最常用的传感器通信接口，IMU（加速度计/陀螺仪）、PPG（心率传感器）、气压计、温湿度传感器等通常都通过 I2C 连接。FarOS 的 I2C 驱动支持主模式和多主模式，最高支持 3.4MHz 高速模式。

I2C 总线配置参数：

参数	选项	适用场景
速度模式	标准（100KHz）、快速（400KHz）、高速（3.4MHz）	传感器通常用 400KHz
地址宽度	7位、10位	绝大多数传感器使用7位地址
传输模式	写、读、组合（写后读）	组合模式常用于读取寄存器

```
#include "i2c.h"

/* 准备I2C传输数据 */
i2c_data_t data = {
    .send_buf = tx_buf,         /* 发送缓冲区 */
    .send_len = tx_len,         /* 发送字节数 */
    .receive_buf = rx_buf,      /* 接收缓冲区 */
    .receive_len = rx_len,      /* 接收字节数 */
};

/* 执行I2C写操作：总线0，设备地址0x50 */
errcode_t ret = uapi_i2c_master_write(I2C_BUS_0, 0x50, &data);

/* 执行I2C读操作 */
ret = uapi_i2c_master_read(I2C_BUS_0, 0x50, &data);

/* 组合传输：先写寄存器地址，再读取数据（常用模式） */
ret = uapi_i2c_master_writeread(I2C_BUS_0, 0x50, &data);
```

5.2 GPIO驱动

GPIO（General Purpose Input/Output）是控制外设和检测外部事件的基础接口。在运动健康设备中，GPIO 用于传感器中断输入、LED指示、按键检测、传感器使能控制等。FarOS 的 GPIO 驱动支持中断触发模式，可实现事件驱动的低功耗设计。

GPIO 工作模式：

模式	功能	应用示例
输出推挽	驱动外部电路	LED控制、传感器使能
输出开漏	多设备共享总线	I2C SDA/SCL（需上拉）
输入浮空	检测外部电平	按键检测（需外部上拉/下拉）

模式	功能	应用示例
输入上拉/下拉	内置电阻稳定电平	中断引脚，防止浮空误触发
中断输入	事件触发回调	传感器数据就绪中断

```
#include "gpio.h"

/* 配置GPIO为输出模式：用于控制LED或传感器使能 */
uapi_gpio_set_dir(GPIO_0, GPIO_DIRECTION_OUTPUT);
uapi_gpio_set_output_val(GPIO_0, GPIO_LEVEL_HIGH);  /* 输出高电平 */

/* 配置GPIO为输入模式：用于读取传感器中断状态 */
uapi_gpio_set_dir(GPIO_1, GPIO_DIRECTION_INPUT);
gpio_level_t level;
uapi_gpio_get_input_val(GPIO_1, &level);  /* 读取当前电平 */

/* 配置GPIO中断：传感器数据就绪时触发回调 */
void sensor_irq_handler(gpio_id_t id, gpio_level_t level)
{
    /* 读取传感器FIFO数据 */
}
uapi_gpio_register_interrupt(GPIO_2, GPIO_INTERRUPT_RISING_EDGE, sensor_irq_handler);
```

5.3 ADC驱动

ADC（Analog-to-Digital Converter）用于将模拟信号转换为数字值。在运动健康设备中，ADC 主要用于读取 PPG（光电容积脉搏波）传感器的原始模拟信号、电池电压监测、温度检测等。FarOS 的 ADC 驱动支持单次采样、连续扫描和自动触发模式。

ADC 采样模式：

模式	说明	适用场景
单次采样	软件触发一次转换	电池电压检测、温度读取
连续扫描	按顺序扫描多个通道	多传感器轮询采集
自动触发	由定时器或DMA触发	PPG高频连续采样

```
#include "adc.h"

/* 配置ADC自动扫描：扫描1个通道 */
adc_scan_config_t config = {
    .type = ADC_SCANNING_MODE_AUTO,      /* 自动扫描模式 */
    .channel_count = 1,                   /* 扫描通道数 */
    .channel_id = { ADC_CHANNEL_0 },      /* 通道0：PPG信号 */
    .sample_rate = ADC_SAMPLE_RATE_1KHZ, /* 采样率1KHz */
};

/* 启动自动扫描 */
uapi_adc_auto_scan_config(&config);

/* 读取采样结果 */
uint16_t adc_value;
uapi_adc_read_sample(ADC_CHANNEL_0, &adc_value);

/* 停止扫描 */
uapi_adc_auto_scan_stop();
```

6. 蓝牙开发指南

FarOS 内置 **BTS (Bluetooth Stack)** 协议栈，完整支持 BLE 5.3 和经典蓝牙（BREDR）协议。蓝牙是运动健康设备与手机 APP、耳机、心率带等外设通信的核心通道，负责数据同步、固件升级、实时数据广播等功能。

BTS 协议栈架构：

```
+-----------------------------------------+
|  Application Profiles                   |
|  GATT / GAP / A2DP / HFP / SPP / AVRCP  |
+-----------------------------------------+
|  Host Stack                             |
|  ATT / SMP / L2CAP / HCI                |
+-----------------------------------------+
|  Controller (Brandy SoC集成)             |
|  Link Layer / PHY / RF                  |
+-----------------------------------------+
```

蓝牙在运动健康设备中的典型应用场景：

场景	协议	说明
数据同步	BLE GATT	运动记录、睡眠数据同步到手机APP
实时心率广播	BLE HRS	向连接的设备实时发送心率数据
固件升级	BLE OTA	通过蓝牙无线升级设备固件
音频播放	A2DP	连接蓝牙耳机播放运动指导语音
通话控制	HFP	蓝牙耳机通话、来电提醒
传感器外设	BLE Client	连接外部心率带、功率计、踏频器等

6.1 BLE GATT服务开发

GATT（Generic Attribute Profile）是 BLE 设备间数据交换的标准协议。FarOS 支持 GATT Server（被连接方，如智能手表）和 GATT Client（主动连接方）两种角色。对外头文件在 include/middleware/services/bts/ble/：

头文件	作用
bts_gatt_server.h	GATT Server：注册服务、添加特征、通知/指示
bts_gatt_client.h	GATT Client：发现服务、读写对端特征
bts_gatt_stru.h	特征属性位、权限位和 GATT 错误码
bts_le_gap.h	设备名、广播参数、扫描和连接
bts_def.h	bt_uuid_t、bd_addr_t 和蓝牙错误码 errcode_bt_t

GATT 基本概念：

概念	说明	类比
Service	一组相关 Characteristic 的集合	数据库中的表
Characteristic	具体的数据项，可读写或通知	表中的字段
Descriptor	Characteristic 的附加属性	字段的元数据
UUID	唯一标识符，区分不同的 Service/Characteristic	主键

bt_uuid_t 用 uuid_len 区分 16-bit UUID（2 字节）和 128-bit UUID（16 字节），内容放在 uuid[]。

特征属性使用 gatt_characteristic_property_t 的位掩码，可以按位组合：

宏	值	含义
GATT_CHARACTER_PROPERTY_BIT_READ	0x02	允许读
GATT_CHARACTER_PROPERTY_BIT_WRITE_NO_RSP	0x04	写且不需要响应

宏	值	含义
GATT_CHARACTER_PROPERTY_BIT_WRITE	0x08	写且需要响应
GATT_CHARACTER_PROPERTY_BIT_NOTIFY	0x10	通知（无需对端确认）
GATT_CHARACTER_PROPERTY_BIT_INDICATE	0x20	指示（需要对端确认）

权限使用 gatt_attribute_permission_t：GATT_ATTRIBUTE_PERMISSION_READ（0x01）、GATT_ATTRIBUTE_PERMISSION_WRITE（0x02），以及加密、认证、授权、MITM 等位。心率等敏感特征应加上 GATT_ATTRIBUTE_PERMISSION_ENCRYPTION_NEED。

GATT Server API：

函数	作用
gatts_register_callbacks(func)	注册添加服务、读写请求、MTU 变化等回调，类型为 gatts_callbacks_t
gatts_register_server(app_uuid, &server_id)	注册 Server，返回 server_id
gatts_add_service_sync(server_id, uuid, is_primary, &handle)	同步添加 Service，直接拿到句柄
gatts_add_characteristic_sync(...)	同步添加 Characteristic，属性填在 gatts_add_chara_info_t
gatts_add_descriptor_sync(...)	同步添加 Descriptor，通知特征需要 CCCD（0x2902）
gatts_start_service(server_id, service_handle)	启动服务，之后才能被对端发现
gatts_send_response(...)	回复读/写请求
gatts_notify_indicate(...)	按句柄发送通知或指示
gatts_set_mtu_size(server_id, mtu)	设置本端 MTU。运动记录建议协商到 185 以上，减少分片

异步版本是 gatts_add_service()、gatts_add_characteristic()、gatts_add_descriptor()，句柄在对应回调里返回。应用代码推荐使用 _sync 版本，避免在回调里拼装数据库。

开发 GATT Server 的调用顺序：

```
#include "bts_gatt_server.h"
#include "bts_le_gap.h"

/* 1. 注册回调，再注册 Server */
gatts_register_callbacks(&g_gatts_cb);
gatts_register_server(&app_uuid, &server_id);

/* 2. 添加 Service 和 Characteristic，然后启动 */
gatts_add_service_sync(server_id, &service_uuid, true, &service_handle);
gatts_add_characteristic_sync(server_id, service_handle, &chara, &result);
gatts_start_service(server_id, service_handle);

/* 3. 设置设备名和广播，等待手机连接 */
gap_ble_set_local_name(name, name_len);
gap_ble_set_adv_param(adv_id, &adv_param);
gap_ble_set_adv_data(adv_id, &adv_data);
gap_ble_start_adv(adv_id);

/* 4. 连接建立后，用特征句柄主动上报 */
gatts_notify_indicate(server_id, conn_id, &ntf);
```

读请求进 gatts_read_request_callback，写请求进 gatts_write_request_callback。回调里填写 gatts_send_response()，不要在回调中做长时间的算法计算。心率、步数这类周期数据放到健康服务回调或 health_task 里，再调用 gatts_notify_indicate()。

作为中心设备连接心率带时，使用 bts_gatt_client.h：gap_ble_start_scan() 发现设备，gap_ble_connect_remote_device() 建立连接，再发现对端的 HRS Service。

6.2 标准健康Profile

FarOS 计划实现以下标准蓝牙健康 Profile，确保与主流运动健康 APP（如 Keep、华为运动健康、Apple Health）的兼容性：

Profile	全称	用途	数据内容	开发状态
HRS	Heart Rate Service	心率数据广播	当前心率、能量消耗、RR间期	待实现
RSCS	Running Speed and Cadence	跑步速度和步频	瞬时速度、步频、总距离	待实现
CSCS	Cycling Speed and Cadence	骑行速度和踏频	踏频、轮速、总里程	待实现
BAS	Battery Service	电池电量	当前电量百分比，上限 BLE_BAS_BATTERY_LEVEL_MAX（100），默认 85	已实现
DIS	Device Information	设备信息	厂商名、型号、固件版本	已实现
CTS	Current Time Service	当前时间同步	年/月/日/时/分/秒	待实现

已实现的示例代码：

Profile	路径
BAS Server	protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/include/ble_bas_server.h
DIS Server	protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/dis_profile_server.c
Find Me	protocol/bt/host/bt/bts/bt_sample/ble_profile/ble_at_sample/src/sample_ble_fmp.c

HRS、RSCS、CSCS、CTS 在协议栈日志和 HCI 事件里有预留标识，仓库中还没有对应的 Profile 实现。自定义健康 Service 也尚未分配 128-bit UUID，接入手机前需要先在 bts_gatt_server.h 的流程里建库。

FarOS 自定义 GATT Service（规划）：

除标准 Profile 外，FarOS 还定义了自定义 Service 用于传输专有数据：

自定义Service	用途	数据类型
FarOS Health Data	运动记录同步	结构化运动数据（JSON/Binary）
FarOS Sleep Data	睡眠记录同步	睡眠分期、心率曲线
FarOS Config	设备配置	用户档案、闹钟、提醒设置
FarOS OTA	固件升级	固件分片、校验、升级状态

7. 运动健康功能开发

本章为手册核心章节，详细介绍 FarOS 运动健康服务模块的架构设计、接口规范和调用方式。参考 Far Beat 运动健康分析框架设计。

7.1 模块架构

运动健康服务模块采用**分层架构**设计，各层职责清晰、接口解耦：

```
+-------------------------------------------------+
|  Application / AT Command Layer                 |
|  - 健康数据展示、运动记录管理、调试命令          |
+-------------------------------------------------+
|  Health Service API Layer                       |
|  - far_health_common    公共类型与全局配置      |
|  - far_activity         运动识别（计步/活动类型）|
|  - far_physio           生理指标（HR/HRV/卡路里等）|
|  - far_sleep            睡眠分析                |
|  - far_health_data      数据管理                |
+-------------------------------------------------+
|  Sensor Abstraction Layer                       |
|  - far_sensor           IMU/PPG传感器抽象       |
+-------------------------------------------------+
|  Driver Layer (I2C/SPI/GPIO/ADC)                |
+-------------------------------------------------+
```

7.1.1 模块依赖关系

模块	依赖模块	说明
far_sensor	驱动层	直接访问硬件传感器
far_activity	far_sensor	使用IMU数据识别活动
far_physio	far_sensor	使用PPG数据计算生理指标
far_sleep	far_sensor, far_physio	使用IMU+HR数据分析睡眠
far_health_data	文件系统/NV	存储和查询健康记录

7.1.2 编译控制

运动健康服务通过 SUPPORT_HEALTH_SERVICE 宏控制编译：

```
/* 在 build/config/target_config/brandy/config.py 中添加：
 * 'SUPPORT_HEALTH_SERVICE': 'YES'
 */
```

brandy-mcu-bt 的 ram_component 目前没有运动健康服务。middleware/services/health/CMakeLists.txt 生成的静态库名是 health_service。打开 SUPPORT_HEALTH_SERVICE 之后，还要把这个库加入目标链接。只改宏、不链接库时，会出现 undefined reference to uapi_health_*。

7.1.3 服务启动与数据通路

对外只有一个总入口：uapi_health_service_init()，实现在 middleware/services/health/health_service_init.c。AT^HEALTHINIT 直接调用它。重复调用时若 initialized 已为真，函数直接返回 ERRCODE_SUCC。

初始化按固定顺序执行，默认参数如下：

步骤	调用	默认参数
1	uapi_health_data_init(FAR_STORAGE_MODE_DUAL)	NV 与文件系统双写
2	uapi_sensor_imu_init()	加速度和陀螺仪都是 50 Hz，标准模式，加速度量程 8 g，陀螺仪量程 2000 dps
3	uapi_sensor_ppg_init()	25 Hz，标准模式，LED 电流 25 mA，只打开绿光
4	注册 IMU/PPG 回调	IMU 回调把样本同时送给运动识别和睡眠分析

步骤	调用	默认参数
5	uapi_activity_init()	计步、活动类型、步频、距离全部打开；步态阈值 150 mg；置信度阈值 70%
6	uapi_physio_init()、uapi_physio_hr_init()	生理指标和心率监测
7	uapi_sleep_init()	自动检测，入睡/起床时刻 255（自动），短于 30 分钟不记，打开 HRV 分析
8	osal_kthread_create()	任务名 health_task，栈 0x1000（4 KB），优先级 18，低于蓝牙协议栈任务
9	使能	IMU（类型 0）、PPG（类型 1）、运动识别、睡眠分析、压力和身体电量

health_task 每 100 ms 醒来一次，按最多 16 帧批量调用 uapi_sensor_imu_read()，再把读到的帧送给 uapi_activity_feed_imu_data() 和 uapi_sleep_feed_imu_data()。IMU 回调里也会做同样的投喂，因此查询步数前不需要应用自己读传感器。

uapi_health_service_deinit() 先停任务，再按睡眠、心率、生理指标、运动识别、传感器回调、IMU、PPG、数据管理的顺序反初始化。

当前源码里下面几步还是空实现，调用相关查询接口前需要知道这一点：

通路	现状
PPG 回调	回调已注册，尚未从原始 ADC 提取心率，也未调用 uapi_physio_feed_hr_data() 和 uapi_sleep_feed_hr_data()
周期算法	任务循环里预留了步频（10 秒）、活动类型（3 秒）、卡路里、HRV（5 分钟）、压力（3 分钟）、身体电量（5 分钟）、睡眠分期（30 秒），函数体尚未填充
落盘	预留了每小时写每日汇总、运动结束写记录、睡眠结束写记录
用户档案	uapi_health_set_user_profile() 校验空指针后返回成功，尚未写入 NV；uapi_health_get_user_profile() 返回清零结构
全局回调	uapi_health_register_callback() 保存前的分发逻辑尚未接到各子模块

心率、HRV、卡路里要得到非零结果，应用需要自行调用 uapi_physio_feed_hr_data() 和 uapi_physio_feed_activity_data()，或先补上 PPG 回调里的提取逻辑。

7.1.4 公共接口与用户档案

公共接口声明在 far_health_common.h，版本号宏是 FAR_HEALTH_API_VERSION_MAJOR/MINOR/PATCH，当前为 1.0.0。

函数	作用
uapi_health_service_init() / uapi_health_service_deinit()	按上一节的顺序启动或关闭全部子模块
uapi_health_get_version(&major, &minor, &patch)	读取 API 版本
uapi_health_set_user_profile(profile)	设置性别、年龄、身高、体重、最大心率和静息心率
uapi_health_get_user_profile(profile)	读取当前档案
uapi_health_register_callback(cb)	注册全局更新回调，类型为 far_health_callback_t
uapi_health_unregister_callback()	注销全局回调

回调的 type 取值见 far_health_data_type_t，例如 FAR_HEALTH_DATA_TYPE_STEP_COUNT（0x01）、FAR_HEALTH_DATA_TYPE_HEART_RATE（0x02）、FAR_HEALTH_DATA_TYPE_SLEEP_STAGE（0x04）。完整枚举见附录 C.9。

卡路里使用 Harris-Benedict 公式，依赖档案里的性别、年龄、身高和体重。weight_kg 的单位是 0.1 kg，70.5 kg 要写成 705。max_hr 为 0 时按 220 - age 计算。

```
#include "far_health_common.h"

far_user_profile_t profile = {
    .gender = 1,          /* 1=男，0=女 */
    .age = 30,
```

```
    .height_cm = 175,
    .weight_kg = 705,      /* 70.5 kg */
    .max_hr = 0,           /* 自动计算 */
    .rest_hr = 58,
};

errcode_t ret = uapi_health_service_init();
if (ret == ERRCODE_SUCC) {
    (void)uapi_health_set_user_profile(&profile);
}
```

档案目前不会跨重启保留。需要掉电保存时，应用应在设置成功后自行写入 NV，启动时再读回并调用 uapi_health_set_user_profile()。

7.2 传感器数据采集

7.2.1 传感器抽象层 (far_sensor)

传感器抽象层提供统一的 IMU（加速度计+陀螺仪）和 PPG（光电容积脉搏波）传感器访问接口，屏蔽底层硬件差异。

核心数据结构：

```
/* IMU数据采样 */
typedef struct {
    int16_t  ax, ay, az;        /* 加速度，单位：mg */
    int16_t  gx, gy, gz;        /* 角速度，单位：mdps */
    uint32_t timestamp;         /* 时间戳，单位：ms */
} far_sensor_imu_data_t;

/* PPG原始数据采样 */
typedef struct {
    int32_t  green_raw;         /* 绿光LED原始ADC值 */
    int32_t  ir_raw;            /* 红外LED原始ADC值 */
    int32_t  red_raw;           /* 红光LED原始ADC值 */
    uint32_t timestamp;         /* 时间戳，单位：ms */
} far_sensor_ppg_data_t;
```

采样率和工作模式是枚举，不是裸数字：

枚举	取值	含义
FAR_SENSOR_RATE_12_5_HZ … FAR_SENSOR_RATE_400_HZ	0–5	12.5 / 25 / 50 / 100 / 200 / 400 Hz
FAR_SENSOR_MODE_LOW_POWER	0	低功耗，精度下降
FAR_SENSOR_MODE_NORMAL	1	标准模式，服务初始化使用这个值
FAR_SENSOR_MODE_HIGH_PERF	2	高性能

uapi_sensor_enable() 和 uapi_sensor_set_sample_rate() 的 sensor_type：0 是 IMU，1 是 PPG。功耗估算接口返回值单位是 uA。

API接口：

函数	功能	返回值
uapi_sensor_imu_init(config)	初始化IMU传感器	errcode_t
uapi_sensor_imu_read(data, max, &read)	读取IMU数据	errcode_t
uapi_sensor_imu_register_callback(cb, user)	注册IMU数据回调	errcode_t
uapi_sensor_ppg_init(config)	初始化PPG传感器	errcode_t
uapi_sensor_ppg_read(data, max, &read)	读取PPG数据	errcode_t
uapi_sensor_ppg_register_callback(cb, user)	注册PPG数据回调	errcode_t
uapi_sensor_set_sample_rate(type, rate)	动态设置采样率	errcode_t
uapi_sensor_enable(type, enable)	使能/禁用传感器	errcode_t

函数	功能	返回值
uapi_sensor_get_power_estimate(type, &power)	获取功耗估算	errcode_t

调用示例：

```
#include "far_sensor.h"

/* 1. 初始化IMU传感器 */
far_sensor_imu_config_t imu_cfg = {
    .accel_rate = FAR_SENSOR_RATE_50_HZ,
    .gyro_rate = FAR_SENSOR_RATE_50_HZ,
    .mode = FAR_SENSOR_MODE_NORMAL,
    .accel_range_g = 8,
    .gyro_range_dps = 2000,
};
uapi_sensor_imu_init(&imu_cfg);

/* 2. 初始化PPG传感器 */
far_sensor_ppg_config_t ppg_cfg = {
    .sample_rate = FAR_SENSOR_RATE_25_HZ,
    .mode = FAR_SENSOR_MODE_NORMAL,
    .led_current_ma = 25,
    .enable_green = true,
    .enable_ir = false,
    .enable_red = false,
};
uapi_sensor_ppg_init(&ppg_cfg);

/* 3. 使能传感器 */
uapi_sensor_enable(0, true);  /* IMU */
uapi_sensor_enable(1, true);  /* PPG */

/* 4. 读取数据 */
far_sensor_imu_data_t imu_data[16];
uint32_t read_len = 0;
uapi_sensor_imu_read(imu_data, 16, &read_len);

/* 5. 异步数据流（推荐） */
void my_imu_callback(const void *data, uint32_t len, void *user_data)
{
    const far_sensor_imu_data_t *imu = (const far_sensor_imu_data_t *)data;
    /* 处理IMU数据... */
}
uapi_sensor_imu_register_callback(my_imu_callback, NULL);
```

7.2.2 传感器配置建议

应用场景	IMU采样率	PPG采样率	功耗估算
日常佩戴（计步）	50Hz	25Hz	~2.7mA
运动模式（跑步）	100Hz	50Hz	~5.5mA
睡眠监测	12.5Hz	12.5Hz	~1.1mA
高精度模式	200Hz	100Hz	~10mA

7.3 运动识别算法接口

7.3.1 功能概述

运动识别模块基于 IMU 传感器数据，实现：

- **计步**（Pedometer）：检测步态周期，统计步数
- **活动类型识别**（Activity Recognition）：识别静止/步行/跑步/骑行等活动类型
- **步频计算**（Cadence）：计算每分钟步数（SPM）

· **距离追踪**（Distance）：基于步数和步长估算行走距离

算法原理说明：

```
计步算法（峰值检测法）：
  1. 合成加速度幅值：acc_mag = sqrt(ax² + ay² + az²)
  2. 低通滤波（截止频率~3Hz）
  3. 检测局部峰值，峰值间隔在 300ms-1500ms 之间
  4. 峰值幅度超过自适应动态阈值

活动类型识别：
  1. 每3秒提取特征（均值、方差、主频率）
  2. 基于规则的决策树分类
     - REST:   acc_var < 1000 mg²
     - WALK:   1Hz < peak_freq < 2.2Hz, acc_var < 5000
     - RUN:    2Hz < peak_freq < 3.5Hz, acc_var > 5000
     - CYCLE:  gyro_var > threshold, acc_var < 3000

步长估算：
  step_length_cm = height_cm × (0.415 + 0.002 × (cadence_spm - 100))
```

仓库提供了三段可回放的 IMU 样本，路径在 application/brandy/brandy_standard/test_data/：

文件	标签	用途
imu_sit.csv	sit	静止/久坐，对应 FAR_ACTIVITY_TYPE_REST
imu_walk.csv	walk	步行，对应 FAR_ACTIVITY_TYPE_WALK
imu_run.csv	run	跑步，对应 FAR_ACTIVITY_TYPE_RUN

列顺序是 timestamp,ax,ay,az,gx,gy,gz,label，时间戳间隔 10 ms（100 Hz）。加速度数值量级接近 9.8，与 far_sensor_imu_data_t 的 mg 单位不同。回放进 uapi_activity_feed_imu_data() 之前，先把加速度换算成 mg，再把角速度换算成 mdps。

API接口：

运动识别模块提供以下API接口。初始化后，模块会在后台持续处理传感器数据，开发者可以通过查询API获取实时结果，也可以注册回调函数接收活动变化通知。

函数	功能	返回值
uapi_activity_init(config)	初始化运动识别模块，配置计步/活动识别/步频/距离功能开关	errcode_t
uapi_activity_enable()	使能运动识别处理，开始接收IMU数据并执行算法	errcode_t
uapi_activity_disable()	禁用运动识别处理，停止算法计算以节省功耗	errcode_t
uapi_activity_get_step_count(&info)	获取当前步数统计信息（总步数、今日步数）	errcode_t
uapi_activity_reset_step_count()	重置步数计数器（通常在每日0点或用户手动重置时调用）	errcode_t
uapi_activity_get_type(&result)	获取当前识别出的活动类型及置信度	errcode_t
uapi_activity_get_cadence(&info)	获取当前步频（SPM）、平均步频和最大步频	errcode_t
uapi_activity_get_distance(&info)	获取累计距离、今日距离和平均步长	errcode_t
uapi_activity_register_callback(cb, user)	注册活动类型变化回调（如从步行变为跑步时触发）	errcode_t
uapi_activity_get_daily_summary(&steps, &dist, &min)	获取当日活动汇总（步数、距离、活跃分钟）	errcode_t
uapi_activity_feed_imu_data(data, len)	投喂一批 IMU 样本。health_task 和 IMU 回调都会调用它	errcode_t
uapi_activity_deinit()	反初始化运动识别模块	errcode_t

调用示例：

```
#include "far_activity.h"

/* 初始化运动识别 */
far_activity_config_t cfg = {
    .enable_step_count = true,
```

```
    .enable_activity_type = true,
    .enable_cadence = true,
    .enable_distance = true,
    .step_threshold = 150,
    .confidence_threshold = 70,
};
uapi_activity_init(&cfg);
uapi_activity_enable();

/* 注册活动变化回调 */
void on_activity_change(far_activity_type_t old_type,
                        far_activity_type_t new_type,
                        uint8_t confidence, void *user_data)
{
    const char *names[] = {"UNKNOWN","REST","WALK","RUN","CYCLE"};
    PRINT("Activity changed: %s -> %s (confidence=%d%%)\r\n",
          names[old_type], names[new_type], confidence);
}
uapi_activity_register_callback(on_activity_change, NULL);

/* 查询步数 */
far_step_info_t steps;
uapi_activity_get_step_count(&steps);
PRINT("Steps today: %lu\r\n", steps.steps_today);

/* 查询距离 */
far_distance_info_t dist;
uapi_activity_get_distance(&dist);
PRINT("Distance today: %lu m\r\n", dist.distance_today_m);
```

7.4 生理指标算法接口（参考Far Beat）

7.4.1 功能概述

生理指标模块参考 **Far Beat** 运动健康分析框架，提供以下高级生理参数。Far Beat 是 FarOS 内置的生理数据分析引擎，融合了运动科学、生理信号处理和机器学习技术，旨在为可穿戴设备提供精准、实时的健康监测能力：

指标	Far Beat概念	算法基础	应用场景
心率监测	Heart Rate	PPG信号峰值检测	实时心率显示
HRV分析	HRV Analysis	RR间期时域/频域分析	自主神经评估
卡路里	Calorie Consumption	METs代谢当量模型	能量消耗统计
VO2max	VO2max Estimation	心率-速度回归模型	有氧能力评估
训练负荷	Training Load (EPOC)	运动后过量氧耗累积	训练强度评估
恢复时间	Recovery Time	EPOC衰减模型	训练恢复建议
压力指数	Stress Monitoring	日间HRV压力分析	压力管理
身体电量	Body Battery	HRV恢复+活动综合模型	能量管理

7.4.2 Far Beat核心概念参考

EPOC（运动后过量氧耗）模型：

EPOC（Excess Post-exercise Oxygen Consumption，运动后过量氧耗）是 Far Beat 训练负荷计算的核心概念。运动后，身体需要消耗额外氧气来恢复到静息状态，这个额外的氧气消耗量即为 EPOC。Far Beat 通过持续监测心率和活动强度，实时计算 EPOC 累积量，从而量化训练对身体的实际生理负荷。

```
EPOC(t) = EPOC(t-1) + (MET(t) - 1) × k - EPOC(t-1) / τ

其中：
  MET(t) = 当前代谢当量
  k      = 个体校准系数
  τ      = EPOC衰减时间常数（30-60分钟）
```

```
训练负荷 = 单次运动中的EPOC峰值或累积量
```

METs（代谢当量）模型：

```
总卡路里 = BMR + 主动运动卡路里
BMR (Harris-Benedict):
  男: 88.362 + 13.397×体重(kg) + 4.799×身高(cm) - 5.677×年龄
  女: 447.593 + 9.247×体重(kg) + 3.098×身高(cm) - 4.330×年龄
主动运动卡路里 = MET × 体重(kg) × 时间(小时)
MET估算基于活动类型和心率区间
```

Body Battery（身体电量）：

```
BodyBattery(t) = BodyBattery(t-1) + Δcharge - Δdrain

充电（休息/睡眠）：
  Δcharge ∝ HRV恢复程度（HF增加）

放电（活动/压力）：
  Δdrain ∝ EPOC累积 + HR升高幅度

范围：0-100
```

7.4.3 API接口

生理指标模块的API按功能分为六大类。所有 API 均返回 errcode_t 类型错误码，ERRCODE_SUCC 表示调用成功。

心率监测API：

心率监测是生理指标的基础，基于PPG传感器信号实时计算心率值。支持当前心率查询、静息心率统计和心率区间分布分析。

函数	功能	说明
uapi_physio_hr_init()	初始化心率监测	启动PPG采样和心率计算任务
uapi_physio_hr_get_current(&info)	获取当前心率	返回当前BPM、心率区间、信号质量
uapi_physio_hr_get_resting(&bpm)	获取静息心率	需要至少7天历史数据，取睡眠期间最低心率
uapi_physio_hr_get_zone_distribution(&dist)	获取心率区间分布	返回当前会话中各区间的停留时间和占比

HRV分析API：

HRV（心率变异性）反映自主神经系统的功能状态，是评估身体恢复能力和压力水平的重要指标。分析需要至少5分钟的连续RR间期数据。

函数	功能	说明
uapi_physio_hrv_analyze(&result)	执行HRV分析	基于缓存的RR间期数据，输出SDNN/RMSSD/LF/HF等指标
uapi_physio_hrv_feed_rr_data(rr, len)	投喂RR间期数据	每次心率检测后将RR间期存入分析缓冲区

卡路里计算API：

基于Far Beat METs模型，结合用户档案（性别、年龄、身高、体重）和活动数据，计算总能量消耗。

函数	功能	说明
uapi_physio_calorie_get_info(&info)	获取卡路里信息	返回总消耗、主动运动消耗、BMR和当前MET值
uapi_physio_calorie_get_today(&total, &active)	获取今日卡路里	返回今日累计的总卡路里和主动运动卡路里

VO2max估算API：

VO2max（最大摄氧量）是衡量有氧能力的黄金标准。Far Beat通过分析跑步或骑行过程中心率和速度/功率的关系，估算用户的VO2max值。

函数	功能	说明
uapi_physio_vo2max_get(&result)	获取VO2max估算值	返回最近一次有效估算结果
uapi_physio_vo2max_estimate(activity)	启动VO2max估算	开始一次新的估算会话，需持续运动15分钟以上

训练负荷与恢复API：

基于EPOC模型评估训练强度和恢复状态，帮助用户科学安排训练计划，避免过度训练。

函数	功能	说明
uapi_physio_training_load_get(&load)	获取训练负荷	返回当前EPOC、会话负荷、周负荷和负荷区间
uapi_physio_training_load_reset()	重置训练负荷	清除当前会话的EPOC累积
uapi_physio_recovery_time_get(&recovery)	获取恢复时间建议	基于EPOC衰减模型计算建议恢复时长

压力与身体电量API：

通过HRV分析实时监测身体压力状态和能量储备，帮助用户平衡活动与休息。

函数	功能	说明
uapi_physio_stress_get(&stress)	获取压力指数	返回0-100的压力水平，越高表示压力越大
uapi_physio_stress_enable(enable)	使能/禁用压力监测	控制压力计算的开关
uapi_physio_body_battery_get(&battery)	获取身体电量	返回0-100的能量储备水平
uapi_physio_body_battery_enable(enable)	使能/禁用身体电量	控制身体电量追踪的开关
uapi_physio_feed_hr_data(bpm, timestamp_ms)	投喂一次心率	PPG 算法得到 BPM 后调用。服务初始化不会自动调用
uapi_physio_feed_activity_data(type, intensity, duration_sec)	投喂活动强度	供卡路里计算使用。intensity 取值 0-100，duration_sec 为本次持续时间

调用示例：

```
#include "far_physio.h"

/* 初始化 */
uapi_physio_init();
uapi_physio_hr_init();

/* 投喂心率数据（由PPG算法产生） */
uapi_physio_feed_hr_data(72, uapi_tcxo_get_ms());

/* 获取心率 */
far_hr_info_t hr;
uapi_physio_hr_get_current(&hr);
PRINT("HR: %u BPM, Zone: %d\r\n", hr.current_bpm, hr.current_zone);

/* HRV分析（需要至少5分钟连续RR数据） */
far_hrv_result_t hrv;
if (uapi_physio_hrv_analyze(&hrv) == ERRCODE_SUCC) {
    PRINT("HRV: SDNN=%ums, RMSSD=%ums, Score=%u\r\n",
          hrv.sdnn_ms, hrv.rmssd_ms, hrv.hrv_score);
}

/* 获取卡路里 */
far_calorie_info_t cal;
uapi_physio_calorie_get_info(&cal);
PRINT("Calories: total=%lu, active=%lu\r\n", cal.total_kcal, cal.active_kcal);

/* 获取压力指数 */
far_stress_info_t stress;
uapi_physio_stress_get(&stress);
PRINT("Stress: %u/100\r\n", stress.current_stress);
```

```
/* 获取身体电量 */
far_body_battery_t bb;
uapi_physio_body_battery_get(&bb);
PRINT("Body Battery: %u/100\r\n", bb.current_level);
```

7.5 睡眠分析接口

7.5.1 功能概述

睡眠分析模块基于 IMU 运动检测和 PPG 心率/HRV 数据，实现睡眠分期检测和睡眠质量评估。

睡眠分期算法：

```
清醒（Wake）：
  - 高频大幅度运动
  - 心率变异大

REM睡眠：
  - 几乎无运动（肌张力抑制）
  - 心率变异性高（类似清醒但身体不动）

浅睡（Light Sleep）：
  - 偶发微动
  - 心率逐渐降低
  - HRV适中

深睡（Deep Sleep）：
  - 几乎无运动
  - 心率最低且稳定
  - HRV低（副交感神经主导）
```

睡眠评分因素：

因素	权重	目标
总睡眠时长	40%	7-9小时
深睡比例	20%	15-25%
睡眠效率	20%	>85%
觉醒次数	10%	≤2次
HRV恢复	10%	睡眠期间HRV改善

API接口：

睡眠分析模块支持自动和手动两种追踪模式。自动模式下，系统通过监测运动和心率变化自动判断入睡和清醒时刻；手动模式则由应用层主动控制睡眠追踪的开始和结束。

函数	功能	说明
uapi_sleep_init(config)	初始化睡眠分析	配置自动检测、预期睡眠时间、HRV分析开关等
uapi_sleep_enable()	使能睡眠分析	启动后台睡眠监测任务
uapi_sleep_start_tracking()	手动开始睡眠追踪	用户主动进入睡眠模式时调用（如点击"开始睡眠"）
uapi_sleep_stop_tracking()	手动停止睡眠追踪	用户主动结束睡眠时调用
uapi_sleep_get_last_record(&record)	获取最近睡眠记录	返回完整的睡眠记录，包括各分期时长、评分、觉醒次数
uapi_sleep_get_score(&score)	获取睡眠评分	返回0-100的综合睡眠质量评分
uapi_sleep_get_stages(&deep, &light, &rem, &awake)	获取睡眠分期	返回各睡眠阶段的持续时间（分钟）
uapi_sleep_is_sleeping(&is_sleeping)	检查是否在睡眠中	返回当前睡眠状态，用于UI状态显示
uapi_sleep_get_quality_factors(&factors)	获取分项评分	返回时长、深睡、效率、规律和 HRV 五项 0-100 评分
uapi_sleep_register_stage_callback(cb, user)	注册分期变化回调	分期切换时给出新分期、上一分期和上一分期时长
uapi_sleep_feed_imu_data(data, len)	投喂 IMU 样本	health_task 与 IMU 回调已调用
uapi_sleep_feed_hr_data(bpm, timestamp_ms)	投喂心率	睡眠分期使用。PPG 回调目前尚未自动调用

函数	功能	说明
uapi_sleep_disable() / uapi_sleep_deinit()	关闭 / 反初始化	uapi_health_service_deinit() 会调用反初始化

调用示例：

```
#include "far_sleep.h"

/* 初始化 */
far_sleep_config_t cfg = {
    .enable_auto_detect = true,
    .bedtime_hour = 255,        /* 自动检测 */
    .wake_time_hour = 255,      /* 自动检测 */
    .min_sleep_duration_min = 30,
    .enable_hrv_analysis = true,
};
uapi_sleep_init(&cfg);
uapi_sleep_enable();

/* 查询睡眠记录 */
far_sleep_record_t record;
if (uapi_sleep_get_last_record(&record) == ERRCODE_SUCC) {
    PRINT("Sleep: %umin total, score=%u\r\n",
          record.total_sleep_min, record.sleep_score);
    PRINT("  Deep: %umin, Light: %umin, REM: %umin, Awake: %umin\r\n",
          record.deep_sleep_min, record.light_sleep_min,
          record.rem_sleep_min, record.awake_min);
}

/* 查询睡眠评分 */
uint8_t score;
uapi_sleep_get_score(&score);
PRINT("Sleep Score: %u/100\r\n", score);
```

7.6 数据管理接口

7.6.1 功能概述

数据管理模块提供运动记录存储、历史数据查询、每日汇总和数据同步功能。支持 **NV存储** 和 **文件系统** 双模式。

存储架构：

存储模式	用途	数据类型
NV存储	快速访问的小数据	今日汇总、最近7天摘要
文件系统	大数据量长期存储	完整运动记录、睡眠分期
双模式	冗余备份	关键数据同时存NV+文件系统

API接口：

数据管理模块提供完整的记录生命周期管理，从数据写入、查询、导出到删除，覆盖健康数据管理的全流程。所有记录均使用唯一ID标识，支持按类型、日期、时长等条件进行过滤查询。

函数	功能	说明
uapi_health_data_init(mode)	初始化数据管理	选择存储模式（NV/文件系统/双模式）
uapi_health_data_save_workout(&record)	保存运动记录	保存一次完整的运动会话记录，自动生成唯一ID
uapi_health_data_save_sleep(&record)	保存睡眠记录	保存睡眠分析结果
uapi_health_data_get_workouts(&filter, &records, max, &count)	查询运动记录	支持按类型、日期范围、时长过滤查询
uapi_health_data_get_daily_summary(&date, &summary)	获取每日汇总	返回指定日期的步数、距离、卡路里、睡眠等综合统计
uapi_health_data_delete_record(id)	删除记录	按记录ID删除单条记录
uapi_health_data_sync_start(callback, user)	启动数据同步	将本地数据同步到手机APP或云端，支持进度回调

函数	功能	说明
uapi_health_data_export(filename, &filter)	导出记录	将符合条件的记录导出为JSON或CSV文件
uapi_health_data_get_sleep_records(start, end, records, max, &count)	查询睡眠记录	按日期范围读取，单次最多 FAR_HEALTH_MAX_QUERY_RECORDS（50）条
uapi_health_data_delete_before(&date)	按日期清理	删除指定日期之前的记录
uapi_health_data_get_storage_usage(&used, &total)	查询存储占用	AT^HEALTHSTATUS 使用该接口，单位为字节
uapi_health_data_deinit()	反初始化	服务反初始化时调用

记录 ID 字符串最长 FAR_HEALTH_RECORD_ID_MAX_LEN（32）字节。一次睡眠记录最多保存 FAR_HEALTH_MAX_SLEEP_SEGMENTS（32）个分期段。查询过滤结构 far_data_query_filter_t 里，type 或 activity_type 填 0xFF 表示该条件不过滤。

同步回调 far_data_sync_callback_t 的 progress 为 0-100，status 为 0（成功）、1（错误）、2（进行中）。

7.6.2 调用示例

```
#include "far_health_data.h"

uapi_health_data_init(FAR_STORAGE_MODE_DUAL);

far_workout_record_t workout = {0};
strncpy_s(workout.record_id, sizeof(workout.record_id), "run-20261023-001", 17);
workout.type = FAR_RECORD_TYPE_WORKOUT;
workout.activity_type = FAR_ACTIVITY_TYPE_RUN;
workout.start_time.year = 2026;
workout.start_time.month = 10;
workout.start_time.day = 23;
workout.start_time.hour = 7;
workout.duration_sec = 1800;
workout.distance_m = 5000;
workout.calories_kcal = 320;
workout.avg_hr = 148;
uapi_health_data_save_workout(&workout);

far_data_query_filter_t filter = {0};
filter.type = 0xFF;
filter.activity_type = FAR_ACTIVITY_TYPE_RUN;
filter.min_duration_sec = 600;
far_workout_record_t records[8];
uint32_t count = 0;
uapi_health_data_get_workouts(&filter, records, 8, &count);

uint32_t used = 0;
uint32_t total = 0;
uapi_health_data_get_storage_usage(&used, &total);
```

7.7 AT命令调试接口

AT 命令是开发和调试运动健康功能的串口入口。命令字符串注册在 application/wstp/at_service/app/app_cmd_manager.c，处理函数在 app_health_at_process.c。所有命令以 AT^HEALTH 为前缀，成功时打印带 [HEALTH] 前缀的文本。下面的示例按处理函数里的格式串书写，数值会随运行数据变化。

使用方式：

1. 使用串口工具连接开发板，波特率 115200，8N1
2. 发送 AT^HEALTHINIT 初始化运动健康服务
3. 使用查询命令读取实时结果
4. 发送 AT^HEALTHDEINIT 反初始化服务

源码注释里出现过 AT^HEALTHSLEEPSTAGE 和 AT^HEALTHPROFILE，当前命令表没有注册这两条。睡眠分期用 AT^HEALTHSLEEP，用户档案用 uapi_health_set_user_profile()。

系统管理命令：

AT命令	功能	串口输出
AT^HEALTHINIT	初始化运动健康服务	[HEALTH] Init OK 或 [HEALTH] Init FAIL
AT^HEALTHDEINIT	反初始化服务	[HEALTH] Deinit OK
AT^HEALTHVER	获取 API 版本号	[HEALTH] Version: 1.0.0
AT^HEALTHSTATUS	查看传感器功耗和存储	分行打印 IMU/PPG/合计功耗（单位 uA）以及 Storage: used/total bytes

运动数据查询命令：

AT命令	功能	串口输出
AT^HEALTHSTEP	获取步数	[HEALTH] Steps: total=<n>, today=<n>
AT^HEALTHACT	获取当前活动类型	[HEALTH] Activity: WALK, confidence=85%, duration=120s。类型名为 UNKNOWN/REST/WALK/RUN/CYCLE/SWIM/HIKE/ELLIPTICAL/ROWING/STAIR/OTHER
AT^HEALTHCAD	获取步频	[HEALTH] Cadence: current=165, avg=160, max=180 SPM
AT^HEALTHDIST	获取距离	[HEALTH] Distance: total=5000m, today=3200m, stride=75.0cm
AT^HEALTHDAILY	获取当日汇总	[HEALTH] Daily: steps=10000, distance=7500m, active=45min。数据来自 uapi_activity_get_daily_summary()，不是 far_daily_summary_t

生理指标查询命令：

AT命令	功能	串口输出
AT^HEALTHHR	获取心率	[HEALTH] HR: current=72, rest=58, max=190, zone=AEROBIC, quality=95%。区间名为 WARMUP/FAT_BURN/AEROBIC/THRESHOLD/ANAEROBIC
AT^HEALTHHRV	执行 HRV 分析	成功时两行：SDNN/RMSSD，以及 LF/HF/score。LF/HF 由 lf_hf_ratio / 100 打印。数据不足时输出 HRV analysis failed (insufficient data)
AT^HEALTHCAL	获取卡路里	[HEALTH] Calories: total=2150, active=450, BMR=1700, MET=3.5。MET 由 current_met / 10 得到
AT^HEALTHVO2	获取 VO2max	有结果时打印 ml/kg/min、置信度和等级；结果为 0 时输出 not available (need running/cycling data)
AT^HEALTHLOAD	获取训练负荷	[HEALTH] Load: EPOC=125, session=850, weekly=5200, zone=OPTIMAL。区间名为 LOW/OPTIMAL/HIGH/OVERREACHING
AT^HEALTHREC	获取恢复时间	[HEALTH] Recovery: 12h0m, score=75, not recovered。完全恢复时最后一段为 recovered
AT^HEALTHSTR	获取压力指数	[HEALTH] Stress: current=35, avg=42, stressed=30min, rest=120min
AT^HEALTHBAT	获取身体电量	[HEALTH] Body Battery: current=78, high=95, low=45, charging。下降时最后一段为 draining

睡眠与传感器命令：

AT命令	功能	串口输出
AT^HEALTHSLEEP	获取最近睡眠记录	第一行 total/deep/light/REM/awake（分钟），第二行 score 和 awakenings。没有记录时输出 Sleep: no record available
AT^HEALTHSLEEPSCORE	获取睡眠评分	[HEALTH] Sleep Score: 82/100
AT^HEALTHSENSOR=<type>,<enable>	开关传感器	type 0 为 IMU，1 为 PPG；enable 1 为打开，0 为关闭。例如 AT^HEALTHSENSOR=0,1。成功时输出 [HEALTH] Sensor 0 enabled

7.8 性能指标与优化建议

7.8.1 算法性能指标

算法性能指标定义了 FarOS 运动健康服务在真实使用场景下应达到的质量标准。这些指标基于行业通用测试方法和医学设备对比验证，确保算法输出具有实际参考价值。

指标	目标值	测试方法	达标意义
步数检测精度	≥ 95%	佩戴设备行走1000步，与标准计步器对比步数误差	日常计步误差控制在±5%以内
活动识别延迟	≤ 3s	从活动类型改变（如坐→走）到算法输出新类型的耗时	用户切换活动时，UI能及时更新
心率监测精度	±3 BPM	与医用12导联ECG同步采集对比	满足日常健康监测和中等强度运动需求
HRV分析精度	相关系数 ≥ 0.9	与医用Holter同步采集，对比SDNN/RMSSD指标	HRV趋势分析具有医学参考价值
卡路里计算误差	≤ 15%	与间接测热法（金标准）对比总能量消耗	饮食和运动计划制定的可靠依据
VO2max估算误差	≤ 10%	与实验室跑台气体分析实测值对比	有氧能力评估和训练计划制定
睡眠分期准确率	≥ 85%	与PSG多导睡眠监测（金标准）对比分期结果	睡眠质量和睡眠障碍筛查参考

7.8.2 系统资源占用

系统资源占用指标反映了运动健康服务对 Brandy SoC 硬件资源的需求。在可穿戴设备中，内存和功耗是最关键的资源约束，需要在算法精度和资源消耗之间取得平衡。

指标	目标值	说明	设计考虑
算法任务内存	≤ 16KB	运行时堆栈+数据区	堆栈约4KB，数据缓冲约12KB，适配中低端硬件
传感器采样功耗	≤ 1mA @ 50Hz	IMU持续采样	50Hz为标准计步模式，12.5Hz低功耗时约0.3mA
PPG采样功耗	≤ 3mA @ 25Hz	心率持续监测	LED电流25mA为典型值，降低电流可进一步省电
算法处理延迟	≤ 10ms/周期	health_task 周期 100 ms	任务栈为 0x1000（4 KB），单次处理留在 10 ms 内，避免挡住下一次批量读取
存储空间占用	≤ 256KB	健康数据存储	NV存储索引+文件系统元数据，支持7天数据缓存

7.8.3 功耗优化建议

功耗优化是运动健康设备的核心约束。uapi_health_service_init() 的默认配置是 IMU 50 Hz、PPG 25 Hz、任务周期 100 ms、每批最多 16 帧。下面的采样组合需要应用调用 uapi_sensor_set_sample_rate() 和 uapi_sensor_enable() 切换，服务本身还不会按活动类型自动改采样率。

1. 动态采样率调整

根据当前活动类型自动调整传感器采样率，避免在非运动状态下消耗不必要的电量。

活动状态	IMU采样率	PPG采样率	估算功耗	适用场景
静止/久坐	12.5Hz	12.5Hz（间歇）	~0.8mA	办公室、看电视
步行	50Hz	25Hz	~2.7mA	日常走路
跑步	100Hz	50Hz	~5.5mA	跑步、健身
深睡	12.5Hz	关闭	~0.3mA	睡眠监测
高精度运动	200Hz	100Hz	~10mA	运动科学研究

2. 批量数据处理

传感器FIFO支持批量数据存储。每次读取时批量获取多个样本，减少CPU唤醒频率：

- IMU FIFO深度通常为32-512帧，每次批量读取16帧
- 批量读取可将CPU唤醒次数从每秒50次降低到每秒3次
- 配合 tickless 机制，CPU在两次读取之间可进入低功耗模式

3. 间歇性PPG采样

非运动状态下，心率变化较缓，可采用间歇采样策略：

- 每5秒唤醒PPG采样1秒，获取心率后关闭LED
- 相比持续采样，功耗降低约80%
- 适用于日常佩戴和睡眠监测场景

4. 深睡检测优化

检测到用户进入深睡期后，系统进入超低功耗模式：

· 关闭PPG（深睡期心率变化无需实时监测）
· IMU降至12.5Hz（仅检测大幅翻身动作）
· 算法任务周期延长至1秒
· 预估深睡期功耗可降低至0.3mA以下

5. 事件驱动架构

将算法任务从周期性轮询改为事件驱动：

· 传感器FIFO满时触发中断唤醒算法任务
· 活动类型变化时触发回调通知上层
· 避免固定周期定时器造成的无效唤醒

功耗预算参考（24小时典型使用场景）：

时段	场景	功耗	时长	电量消耗
夜间睡眠（8小时）	深睡监测	0.3mA	8h	2.4mAh
日间静止（10小时）	日常佩戴	0.8mA	10h	8.0mAh
日间活动（4小时）	步行/办公	2.7mA	4h	10.8mAh
运动时段（1小时）	跑步	5.5mA	1h	5.5mAh
蓝牙同步（1小时）	数据上传	8.0mA	1h	8.0mAh
24小时总计		**平均1.46mA**	**24h**	**约35mAh**

以200mAh电池容量计算，FarOS运动健康服务可实现约5-6天的续航时间。

8. 故障排除与FAQ

本章汇总 FarOS 开发过程中常见的问题及其解决方案，按问题类型分类，便于快速定位。

8.1 编译问题

问题现象	可能原因	解决方案
cc_riscv32_musl_fp_win: command not found	编译器未添加到 PATH	检查 tools/bin/compiler/linx/linx_170/bin/ 是否已加入系统环境变量
CMake Error: Could not find CMakeLists.txt	未执行配置步骤	先运行 build.py -t brandy-mcu-bt -c 生成构建配置
链接错误：undefined reference to uapi_*	未链接对应的库或模块未使能	检查 CMakeLists.txt 中是否添加了对应模块的依赖，确认 config.py 中相关功能开关为 YES
编译报错：SUPPORT_HEALTH_SERVICE not defined	运动健康服务宏未定义	在 build/config/target_config/brandy/config.py 中添加 'SUPPORT_HEALTH_SERVICE': 'YES'
编译耗时过长	未使用并行编译	添加 -j8 参数启用8线程编译，如 build.py -t brandy-mcu-bt -b -j8

8.2 烧录问题

问题现象	可能原因	解决方案
J-Link not found	调试器未连接或驱动未安装	检查USB连接，安装J-Link驱动，确认设备管理器中识别到J-Link
Failed to connect to target	SWD接线错误或开发板未上电	检查SWD线序（VCC/GND/TCK/TMS），确认开发板电源指示灯亮起
烧录成功但设备无响应	烧录了错误的固件目标	确认 -t 参数与开发板型号匹配，如 brandy-mcu-bt
烧录后反复重启	固件与硬件不匹配或NV数据损坏	尝试擦除整片Flash后重新烧录，或检查板级配置（drivers/boards/）是否正确

8.3 运行时问题

问题现象	可能原因	解决方案
串口无输出	波特率设置错误	确认串口工具波特率为 **115200，8N1**
串口输出乱码	波特率不匹配或线序错误	检查USB转TTL接线（TX/RX交叉连接），确认波特率115200
系统启动后panic重启	栈溢出或内存越界	增大任务栈空间（attr.stack_size），使用 memset_s/memcpy_s 替代不安全的内存操作
任务无响应	优先级设置不当或死锁	检查任务优先级分配，使用 osSemaphoreAcquire() 替代忙等待，确认消息队列未满
功耗异常偏高	传感器未正确关闭或tickless未启用	检查 uapi_sensor_enable() 是否在非使用场景调用了关闭，确认 ENABLE_TICKLESS 配置为 YES

8.4 运动健康模块问题

问题现象	可能原因	解决方案
AT^HEALTHINIT 返回失败	传感器初始化失败或内存不足	检查IMU/PPG的I2C连接，确认 far_sensor.c 中传感器型号与实际硬件匹配
步数统计为0	IMU未使能或算法未启动	确认 uapi_sensor_enable(0, true) 已调用，且 uapi_activity_enable() 已执行
心率显示为0或异常值	PPG 回调尚未把原始 ADC 转成 BPM，或传感器未贴紧、LED 电流过低	调用 uapi_physio_feed_hr_data() 做通路验证；硬件侧检查贴合，并增大 led_current_ma
HRV分析返回失败	RR间期数据不足	确保心率监测已运行至少5分钟，且 uapi_physio_hrv_feed_rr_data() 正常被调用
睡眠记录为空	自动检测未触发或追踪未启动	确认 enable_auto_detect = true，或手动调用 uapi_sleep_start_tracking()
数据同步失败	蓝牙未连接或存储空间满	检查BLE连接状态，使用 AT^HEALTHSTATUS 查看存储使用率，必要时删除旧记录
卡路里计算偏差大	用户档案未设置或活动类型识别错误	调用 uapi_health_set_user_profile() 设置性别/年龄/身高/体重，确认活动类型识别正常

调试技巧：

1. **使用 AT 命令快速诊断**：AT^HEALTHSTATUS 打印 IMU/PPG 功耗（单位 uA）和存储字节数。心率一直为 0 时，先确认是否已调用 uapi_physio_feed_hr_data()，当前 PPG 回调还不会自动提取 BPM
2. **开启DFX详细日志**：在 config.py 中设置 'LOG_LEVEL': 'DEBUG'，查看模块内部运行细节
3. **检查任务状态**：通过 osThreadGetCount() 和 osThreadEnumerate() 查看当前运行的任务列表
4. **内存泄漏排查**：定期打印 osal_mem_get_heap_info() 返回值，观察空闲内存是否持续下降

9. 附录

附录A: 运动健康API速查表

A.1 公共API

函数	文件
uapi_health_service_init()	health_service_init.c
uapi_health_service_deinit()	health_service_init.c
uapi_health_get_version()	health_service_init.c
uapi_health_set_user_profile()	health_service_init.c
uapi_health_get_user_profile()	health_service_init.c
uapi_health_register_callback()	health_service_init.c
uapi_health_unregister_callback()	health_service_init.c

A.2 传感器API

函数	文件
uapi_sensor_imu_init()	far_sensor.c
uapi_sensor_imu_read()	far_sensor.c
uapi_sensor_ppg_init()	far_sensor.c
uapi_sensor_ppg_read()	far_sensor.c
uapi_sensor_set_sample_rate()	far_sensor.c
uapi_sensor_enable()	far_sensor.c

A.3 运动识别API

函数	文件
uapi_activity_init()	far_activity.c
uapi_activity_enable()	far_activity.c
uapi_activity_get_step_count()	far_activity.c
uapi_activity_get_type()	far_activity.c
uapi_activity_get_cadence()	far_activity.c
uapi_activity_get_distance()	far_activity.c
uapi_activity_feed_imu_data()	far_activity.c
uapi_activity_get_daily_summary()	far_activity.c

A.4 生理指标API

函数	文件
uapi_physio_init()	far_physio.c
uapi_physio_hr_get_current()	far_physio.c
uapi_physio_hrv_analyze()	far_physio.c
uapi_physio_calorie_get_info()	far_physio.c
uapi_physio_vo2max_get()	far_physio.c
uapi_physio_training_load_get()	far_physio.c
uapi_physio_recovery_time_get()	far_physio.c
uapi_physio_stress_get()	far_physio.c
uapi_physio_body_battery_get()	far_physio.c
uapi_physio_feed_hr_data()	far_physio.c
uapi_physio_feed_activity_data()	far_physio.c

A.5 睡眠分析API

函数	文件
uapi_sleep_init()	far_sleep.c
uapi_sleep_enable()	far_sleep.c
uapi_sleep_get_last_record()	far_sleep.c
uapi_sleep_get_score()	far_sleep.c
uapi_sleep_get_stages()	far_sleep.c
uapi_sleep_feed_imu_data()	far_sleep.c
uapi_sleep_feed_hr_data()	far_sleep.c
uapi_sleep_get_quality_factors()	far_sleep.c

A.6 数据管理API

函数	文件
uapi_health_data_init()	far_health_data.c
uapi_health_data_save_workout()	far_health_data.c
uapi_health_data_get_daily_summary()	far_health_data.c
uapi_health_data_sync_start()	far_health_data.c
uapi_health_data_get_sleep_records()	far_health_data.c
uapi_health_data_get_storage_usage()	far_health_data.c
uapi_health_data_save_sleep()	far_health_data.c

附录B: 错误码表

FarOS 运动健康服务使用统一的 errcode_t 错误码体系。错误码为32位无符号整数，高4位为模块标识，低28位为具体错误号。调用API后，应始终检查返回值，确保操作成功。

常见错误码及处理建议：

错误码	值	说明	常见原因	处理建议
ERRCODE_SUCC	0x00000000	成功	操作正常完成	无需处理
ERRCODE_FAIL	0xFFFFFFFF	通用失败	硬件异常、资源不足	检查硬件连接，重试操作
ERRCODE_INVALID_PARAM	0x80000001	参数无效	传入空指针、越界值	检查参数合法性

错误码	值	说明	常见原因	处理建议
ERRCODE_NOT_SUPPORT	0x80000002	不支持的操作	功能未实现或目标不支持	确认功能版本，使用替代方案

错误码使用示例：

```
errcode_t ret = uapi_activity_init(&cfg);
if (ret != ERRCODE_SUCC) {
    if (ret == ERRCODE_INVALID_PARAM) {
        PRINT("Error: Invalid configuration parameter\r\n");
    } else if (ret == ERRCODE_FAIL) {
        PRINT("Error: Activity recognition init failed\r\n");
    }
    return ret;
}
```

附录C: 数据类型定义

C.1 活动类型枚举

```
typedef enum {
    FAR_ACTIVITY_TYPE_UNKNOWN   = 0,
    FAR_ACTIVITY_TYPE_REST      = 1,    /* 静止/久坐 */
    FAR_ACTIVITY_TYPE_WALK      = 2,    /* 步行 */
    FAR_ACTIVITY_TYPE_RUN       = 3,    /* 跑步 */
    FAR_ACTIVITY_TYPE_CYCLE     = 4,    /* 骑行 */
    FAR_ACTIVITY_TYPE_SWIM      = 5,    /* 游泳 */
    FAR_ACTIVITY_TYPE_HIKE      = 6,    /* 徒步 */
    FAR_ACTIVITY_TYPE_ELLIPTICAL = 7,   /* 椭圆机 */
    FAR_ACTIVITY_TYPE_ROWING    = 8,    /* 划船 */
    FAR_ACTIVITY_TYPE_STAIR     = 9,    /* 爬楼梯 */
    FAR_ACTIVITY_TYPE_SPORT_OTHER = 10,/* 其他运动 */
} far_activity_type_t;
```

C.2 睡眠分期枚举

```
typedef enum {
    FAR_SLEEP_STAGE_WAKE    = 0,    /* 清醒 */
    FAR_SLEEP_STAGE_REM     = 1,    /* REM快速眼动期 */
    FAR_SLEEP_STAGE_LIGHT   = 2,    /* 浅睡期 */
    FAR_SLEEP_STAGE_DEEP    = 3,    /* 深睡期 */
} far_sleep_stage_t;
```

C.3 心率区间枚举（Karvonen模型）

```
typedef enum {
    FAR_HR_ZONE_WARMUP      = 0,    /* 热身区 < 60% HRR */
    FAR_HR_ZONE_FAT_BURN    = 1,    /* 燃脂区 60-70% HRR */
    FAR_HR_ZONE_AEROBIC     = 2,    /* 有氧耐力区 70-80% HRR */
    FAR_HR_ZONE_THRESHOLD   = 3,    /* 乳酸阈值区 80-90% HRR */
    FAR_HR_ZONE_ANAEROBIC   = 4,    /* 无氧区 90-100% HRR */
} far_hr_zone_t;
```

C.4 用户档案结构体

```
typedef struct {
    uint8_t  gender;        /* 0=女, 1=男 */
    uint8_t  age;           /* 岁 */
    uint16_t height_cm;     /* 厘米 */
    uint16_t weight_kg;     /* 千克×10（0.1kg精度） */
    uint8_t  max_hr;        /* 自定义最大心率，0=自动计算 */
    uint8_t  rest_hr;       /* 实测静息心率 */
} far_user_profile_t;
```

weight_kg 的单位是 0.1 kg。时间戳 far_health_timestamp_t 的字段为 year、month、day、hour、minute、second。

C.5 传感器配置

```
typedef enum {
    FAR_SENSOR_RATE_12_5_HZ = 0,
    FAR_SENSOR_RATE_25_HZ   = 1,
    FAR_SENSOR_RATE_50_HZ   = 2,
    FAR_SENSOR_RATE_100_HZ  = 3,
    FAR_SENSOR_RATE_200_HZ  = 4,
    FAR_SENSOR_RATE_400_HZ  = 5,
} far_sensor_rate_t;

typedef enum {
    FAR_SENSOR_MODE_LOW_POWER = 0,
    FAR_SENSOR_MODE_NORMAL    = 1,
    FAR_SENSOR_MODE_HIGH_PERF = 2,
} far_sensor_mode_t;
```

far_sensor_imu_config_t 含 accel_rate、gyro_rate、mode、accel_range_g、gyro_range_dps。far_sensor_ppg_config_t 含 sample_rate、mode、led_current_ma，以及 enable_green、enable_ir、enable_red。

C.6 运动识别结果

```
typedef struct {
    bool     enable_step_count;
    bool     enable_activity_type;
    bool     enable_cadence;
    bool     enable_distance;
    uint16_t step_threshold;       /* mg，默认 150 */
    uint8_t  confidence_threshold; /* %，默认 70 */
} far_activity_config_t;

typedef struct {
    uint32_t total_steps;
    uint32_t steps_today;          /* 每日 0 点重置 */
    uint32_t last_update_time;     /* ms */
} far_step_info_t;

typedef struct {
    far_activity_type_t type;
    uint8_t             confidence;    /* 0-100 */
    uint32_t            duration_sec;
    uint32_t            start_time;    /* ms */
} far_activity_result_t;
```

far_cadence_info_t 含 current_spm、avg_spm、max_spm。far_distance_info_t 含 total_distance_m、distance_today_m 和 avg_stride_length_cm（单位厘米）。

C.7 生理指标结果

结构体	关键字段
far_hr_info_t	current_bpm、resting_bpm、max_bpm、current_zone、signal_quality（0-100）
far_hrv_result_t	sdnn_ms、rmssd_ms、lf_power、hf_power、lf_hf_ratio（实际比值×100）、hrv_score
far_calorie_info_t	total_kcal、active_kcal、bmr_kcal、current_met（MET×10）
far_vo2max_result_t	vo2max_ml_kg_min、confidence、fitness_level（1-10）
far_training_load_t	current_epoc（ml/kg）、session_load、weekly_load、load_zone（0 低，1 最佳，2 高，3 过度）
far_recovery_info_t	recovery_hours、recovery_minutes、recovery_score、is_recovered
far_stress_info_t	current_stress、avg_stress（0-100）、stress_duration_min、rest_duration_min
far_body_battery_t	current_level、highest_today、lowest_today（0-100）、is_charging、charge_rate

C.8 睡眠记录与存储

```
typedef struct {
    bool     enable_auto_detect;
    uint8_t  bedtime_hour;          /* 0-23, 255=自动 */
    uint8_t  wake_time_hour;        /* 0-23, 255=自动 */
    uint16_t min_sleep_duration_min;
    bool     enable_hrv_analysis;
} far_sleep_config_t;

typedef enum {
    FAR_STORAGE_MODE_NV   = 0,
    FAR_STORAGE_MODE_FS   = 1,
    FAR_STORAGE_MODE_DUAL = 2,
} far_storage_mode_t;

typedef enum {
    FAR_RECORD_TYPE_ACTIVITY     = 0,
    FAR_RECORD_TYPE_WORKOUT      = 1,
    FAR_RECORD_TYPE_SLEEP        = 2,
    FAR_RECORD_TYPE_HRV          = 3,
    FAR_RECORD_TYPE_BODY_BATTERY = 4,
    FAR_RECORD_TYPE_STRESS       = 5,
} far_record_type_t;
```

far_sleep_record_t 记录起止时间戳（ms）、总睡眠和深睡/浅睡/REM/清醒分钟数、觉醒次数、sleep_score，以及最多 32 个 far_sleep_segment_t。far_workout_record_t 保存记录 ID、活动类型、起止时间、步数、距离、卡路里、心率、步频、5 个心率区间停留分钟、VO2max 和训练负荷。far_daily_summary_t 按自然日汇总步数、距离、卡路里、活跃分钟、静息心率、压力、睡眠评分和身体电量高低值。

C.9 健康数据更新类型

far_health_data_type_t 用于 uapi_health_register_callback()：

枚举	值	含义
FAR_HEALTH_DATA_TYPE_STEP_COUNT	0x01	步数更新
FAR_HEALTH_DATA_TYPE_HEART_RATE	0x02	心率更新
FAR_HEALTH_DATA_TYPE_ACTIVITY	0x03	活动类型变化
FAR_HEALTH_DATA_TYPE_SLEEP_STAGE	0x04	睡眠分期变化
FAR_HEALTH_DATA_TYPE_HRV	0x05	HRV 结果更新
FAR_HEALTH_DATA_TYPE_STRESS	0x06	压力指数更新
FAR_HEALTH_DATA_TYPE_BODY_BATTERY	0x07	身体电量更新
FAR_HEALTH_DATA_TYPE_CALORIE	0x08	卡路里更新
FAR_HEALTH_DATA_TYPE_DISTANCE	0x09	距离更新
FAR_HEALTH_DATA_TYPE_VO2MAX	0x0A	VO2max 更新
FAR_HEALTH_DATA_TYPE_TRAINING_LOAD	0x0B	训练负荷更新
FAR_HEALTH_DATA_TYPE_RECOVERY_TIME	0x0C	恢复时间更新

附录D: 文件清单

D.1 头文件

文件路径	说明
include/middleware/services/health/far_health_common.h	公共类型定义
include/middleware/services/health/far_sensor.h	传感器抽象层
include/middleware/services/health/far_activity.h	运动识别
include/middleware/services/health/far_physio.h	生理指标
include/middleware/services/health/far_sleep.h	睡眠分析
include/middleware/services/health/far_health_data.h	数据管理

D.2 服务实现文件（middleware/services/health/）

源文件包含运动健康算法的骨架实现，算法逻辑以详细注释形式说明，后续可根据具体传感器型号和算法需求进行填充优化。

文件路径	说明	核心实现
health_service_init.c	服务初始化与任务	模块总初始化、后台健康任务（100ms调度）、传感器回调注册
far_sensor.c	传感器抽象实现	IMU/PPG初始化、数据读取、采样率设置、功耗估算
far_activity.c	运动识别实现	峰值检测计步、活动分类决策树、步长估算、距离计算
far_physio.c	生理指标实现	心率监测、HRV分析、METs卡路里、VO2max估算、EPOC训练负荷、恢复时间、压力、身体电量
far_sleep.c	睡眠分析实现	睡眠分期检测、睡眠评分、自动入睡/清醒检测
far_health_data.c	数据管理实现	记录存储、历史查询、每日汇总、数据同步、导出
CMakeLists.txt	构建配置	模块编译规则、头文件路径、依赖库链接

D.3 AT命令扩展文件（application/wstp/at_service/app/）

AT命令扩展为运动健康模块提供串口调试能力，便于开发阶段验证算法功能和排查问题。

文件路径	说明	功能
app_health_at_process.h	AT命令头文件	函数声明、接口定义
app_health_at_process.c	AT命令实现	20+个健康AT命令处理函数

D.4 修改的现有文件

运动健康模块通过以下修改集成到现有AT命令框架中：

文件路径	修改内容
app_cmd_manager.c	注册健康AT命令到全局AT命令表
app_msg_manager.h	添加健康相关消息ID枚举

附录E: 术语表

E.1 通用术语

术语	英文全称	说明
SoC	System on Chip	片上系统，将CPU、内存、外设集成在单一芯片上
MCU	Microcontroller Unit	微控制器单元，集成处理器核心、存储器和外设接口的单芯片计算机
RTOS	Real-Time Operating System	实时操作系统，保证任务在规定时间约束内完成响应
HAL	Hardware Abstraction Layer	硬件抽象层，将底层硬件差异封装，向上层提供统一接口
BSP	Board Support Package	板级支持包，包含特定开发板的引脚配置、外设初始化等
DFX	Design For X	面向全生命周期的设计，涵盖可测试性、可制造性、可靠性等
OTA	Over-The-Air	空中升级，通过无线网络远程更新设备固件
NV	Non-Volatile	非易失性存储，断电后数据不丢失
VFS	Virtual File System	虚拟文件系统，为不同底层文件系统提供统一访问接口
FIFO	First In First Out	先进先出队列，常用于传感器数据缓冲
DMA	Direct Memory Access	直接内存访问，无需CPU干预的数据传输方式
CMSIS	Cortex Microcontroller Software Interface Standard	ARM推出的微控制器软件接口标准，FarOS中用于RTOS2接口

E.2 传感器与健康术语

术语	英文全称	说明
IMU	Inertial Measurement Unit	惯性测量单元，通常包含加速度计和陀螺仪
PPG	Photoplethysmography	光电容积脉搏波描记法，通过光学方式检测血容量变化来测量心率
HR	Heart Rate	心率，每分钟心跳次数（BPM）
HRV	Heart Rate Variability	心率变异性，连续心跳间期（RR间期）的波动程度
BPM	Beats Per Minute	每分钟心跳次数
ECG	Electrocardiogram	心电图，记录心脏电活动的医学金标准
MET	Metabolic Equivalent of Task	代谢当量，衡量运动强度的单位，1MET≈静息代谢率
VO2max	Maximal Oxygen Uptake	最大摄氧量，衡量有氧耐力的黄金指标
EPOC	Excess Post-exercise Oxygen Consumption	运动后过量氧耗，运动后恢复期的额外氧气消耗
SPM	Steps Per Minute	每分钟步数，即步频
PSG	Polysomnography	多导睡眠监测，睡眠分期的医学金标准
REM	Rapid Eye Movement	快速眼动睡眠期，梦境主要发生阶段
SDNN	Standard Deviation of NN intervals	RR间期的标准差，HRV时域指标
RMSSD	Root Mean Square of Successive Differences	相邻RR间期差值的均方根，HRV时域指标
LF/HF	Low Frequency / High Frequency	HRV频域分析中的低频/高频功率比，反映交感/副交感神经平衡
TRNG	True Random Number Generator	真随机数生成器，基于物理熵源产生不可预测的随机数

E.3 蓝牙术语

术语	英文全称	说明
BLE	Bluetooth Low Energy	低功耗蓝牙，适合小数据量、低功耗的短距离通信
BREDR	Basic Rate / Enhanced Data Rate	经典蓝牙，支持较高数据速率和音频传输
GATT	Generic Attribute Profile	通用属性协议，BLE设备间数据交换的基础协议
GAP	Generic Access Profile	通用访问协议，定义BLE设备的发现、连接流程
UUID	Universally Unique Identifier	通用唯一识别码，用于标识蓝牙服务和特征值
Profile	-	蓝牙规范定义的特定应用场景协议（如HRS心率服务、A2DP音频传输）
HCI	Host Controller Interface	主机控制器接口，定义蓝牙主机与控制器间的通信协议
L2CAP	Logical Link Control and Adaptation Protocol	逻辑链路控制与适配协议，蓝牙数据链路层
SMP	Security Manager Protocol	安全管理协议，负责BLE配对和密钥分发
Coded PHY	Coded Physical Layer	编码物理层，BLE 5.0+特性，通过编码增益实现更远传输距离

版本历史

版本	日期	修改内容
v1.2	2026-10-23	按源码补充源码目录、启动顺序、编译目标、GATT Server API、健康服务初始化与数据通路、用户档案和数据管理示例；AT 输出与实现格式对齐；补齐附录数据类型；修正第 6 章标题编号
v1.1	2026-10	新增快速开始指南、故障排除与FAQ章节、术语表；修复标题层级一致性；完善系统架构图
v1.0	2024-10	初始版本，添加运动健康服务模块完整接口规范和骨架代码

本文档为 FarOS 运动健康微内核操作系统开发手册，版权归 国体智慧体育技术创新中心 所有。